

Kubernetes 详解

王天青 | 2023-06

目录 >

CONTENTS

- 1 Pod 深入
- 2 K8s 设计理念
- 2 K8s 工作机制



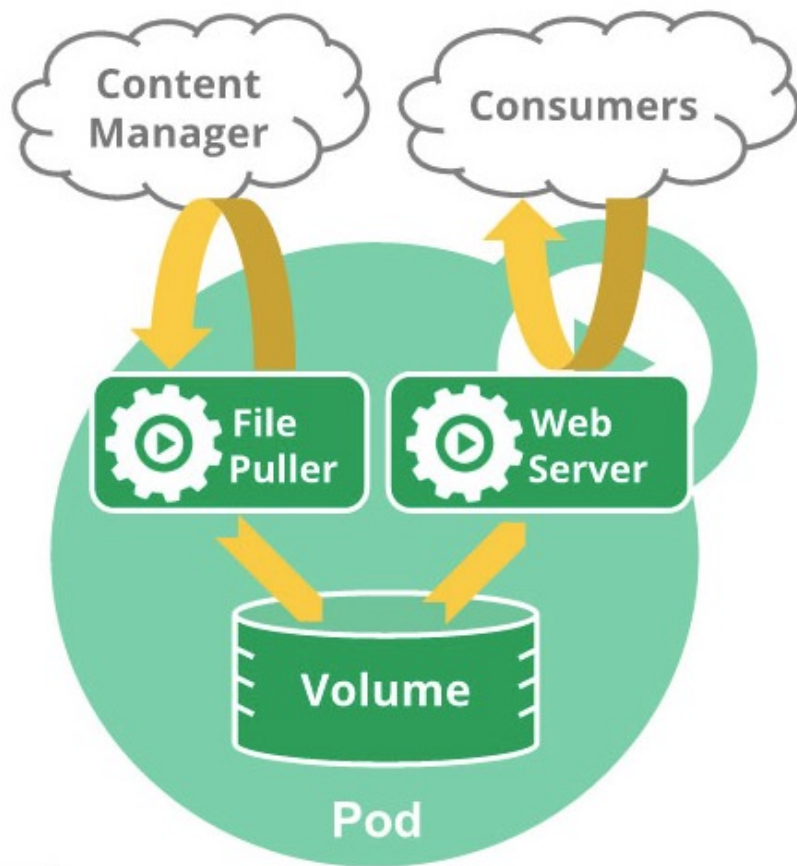
1 chapter

Pod 深入

- ✓ Pod 回顾
- ✓ Pod 核心配置
- ✓ Pod 生命周期

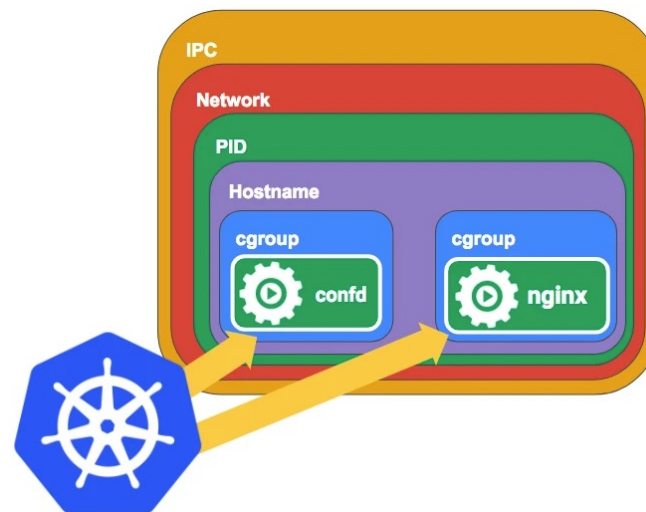
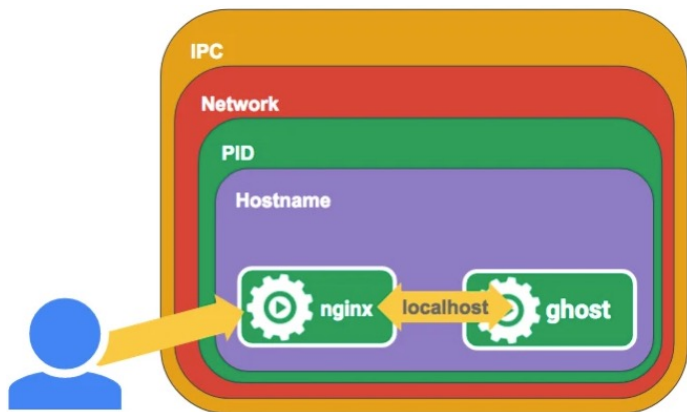
➤ Pod

- 最小的管理元素不是一个个独立的容器，而是Pod；
- Pod是最小的管理、创建、计划的最小单元。



➤ Pod详解

- 一个Pod（就像一群鲸鱼，或者一个豌豆夹）相当于一个共享 context 的配置组，在同一个 context 下，应用可能还会有独立的 cgroup 隔离机制。
- Pod 的 context 可以理解成多个 linux 命名空间的结合
 - PID 命名空间（同一个 Pod 中应用可以看到其它进程）
 - 网络 命名空间（同一个 Pod 的中的应用对相同的 IP 地址和端口有权限）
 - IPC 命名空间（同一个 Pod 中的应用可以通过VPC 或者 POSIX 进行通信）
 - UTS 命名空间（同一个 Pod 中的应用共享一个主机名称）



➤ Pod 详解

- Pod 资源共享

- 存储共享
- 通信共享
- 网络共享

- Pod 管理

- Pod 通过提供一个高层次抽象而不是底层的接口简化了应用的部署及管理。
- 容器探针

- Pod 使用

- 通常不需要直接创建 Pod，甚至单实例 Pod。
- Deployment
- StatefulSet
- DaemonSet

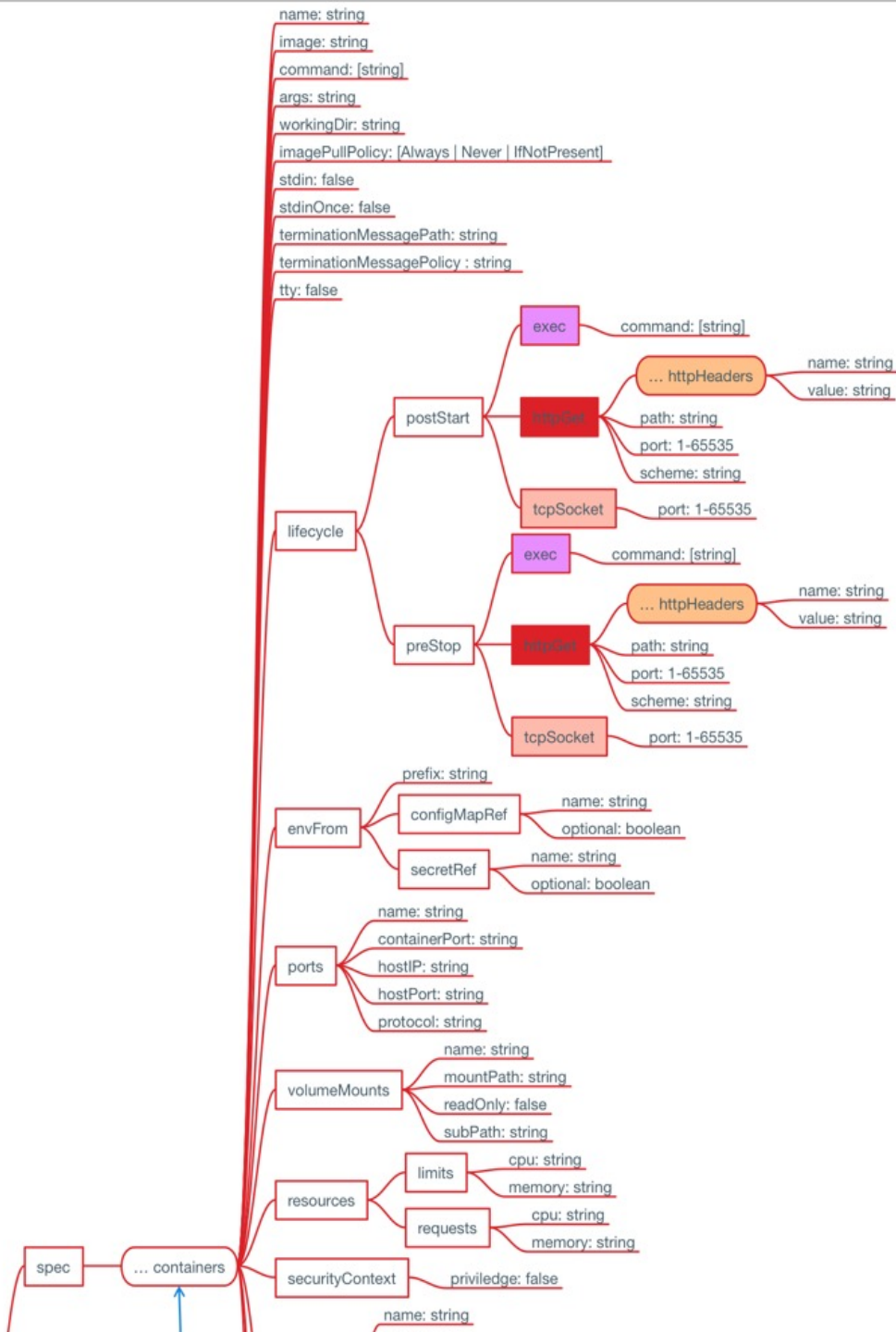
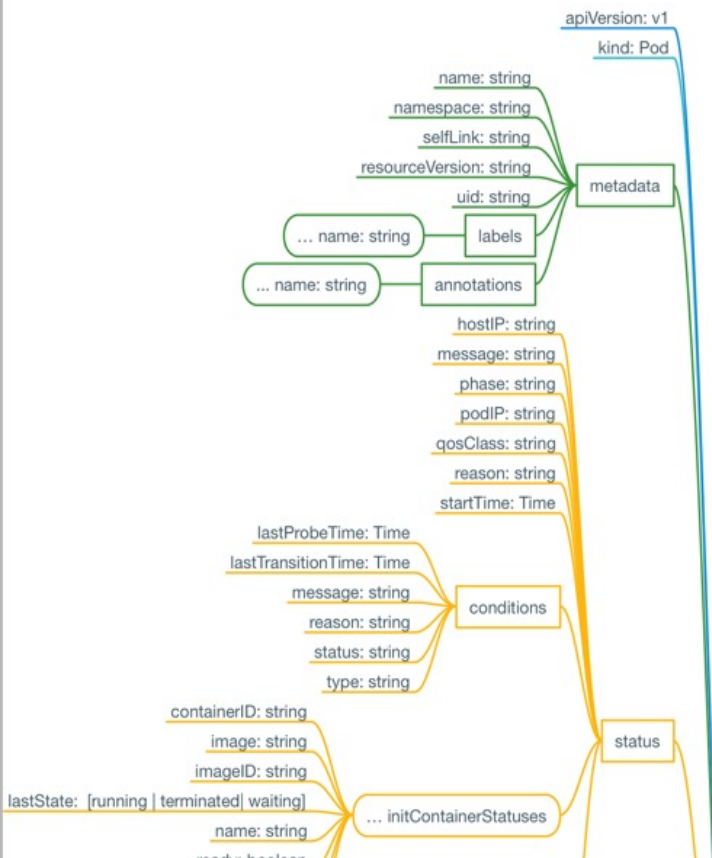
➤ Pod 详解

- Kubernetes 集群中的 Pod 主要有两种用法：
 - 运行单个容器的 Pod。"每个 Pod 一个容器"模型是最常见的 Kubernetes 用例；在这种情况下，可以将 Pod 看作单个容器的包装器，并且 Kubernetes 直接管理 Pod，而不是容器。
 - 运行多个协同工作的容器的 Pod。Pod 可能封装由多个紧密耦合且需要共享资源的共处容器组成的应用程序。这些位于同一位置的容器可能形成单个内聚的服务单元——一个容器将文件从共享卷提供给公众，而另一个单独的“边车”（sidecar）容器则刷新或更新这些文件。Pod 将这些容器和存储资源打包为一个可管理的实体。
 - **说明：** 将多个并置、同管的容器组织到一个 Pod 中是一种相对高级的使用场景。只有在一些场景中，容器之间紧密关联时你才应该使用这种模式。

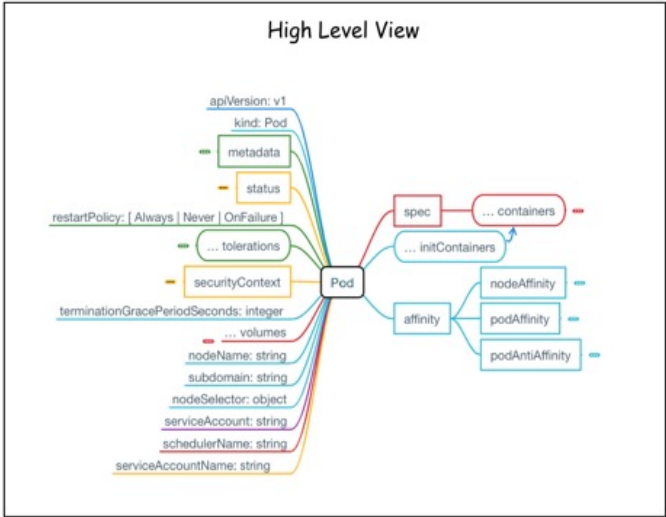
Pod Cheat Sheet

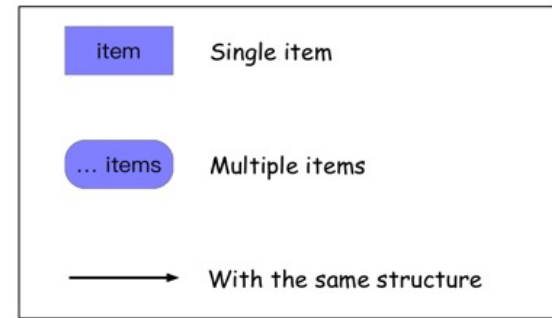
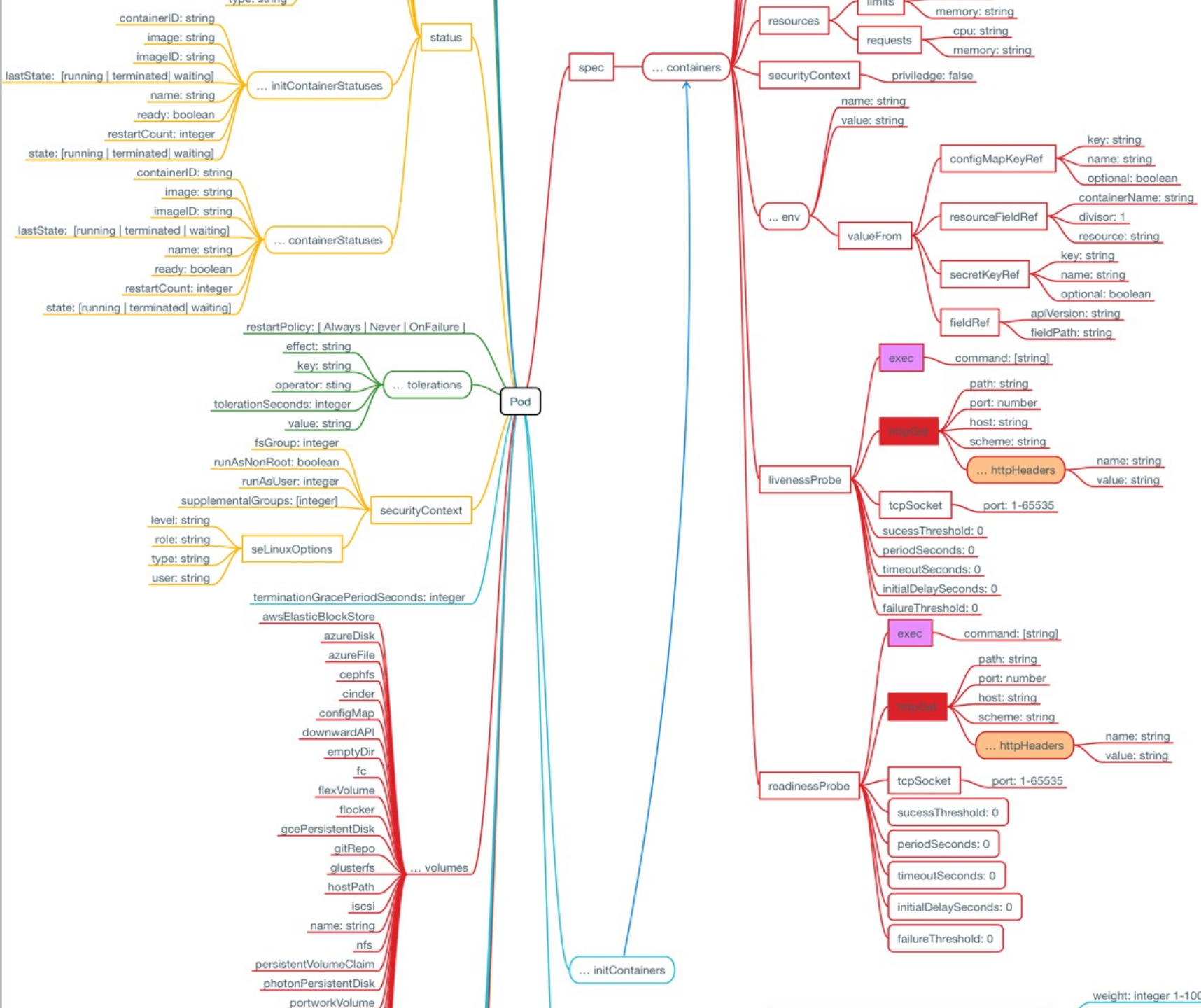


kubernetes



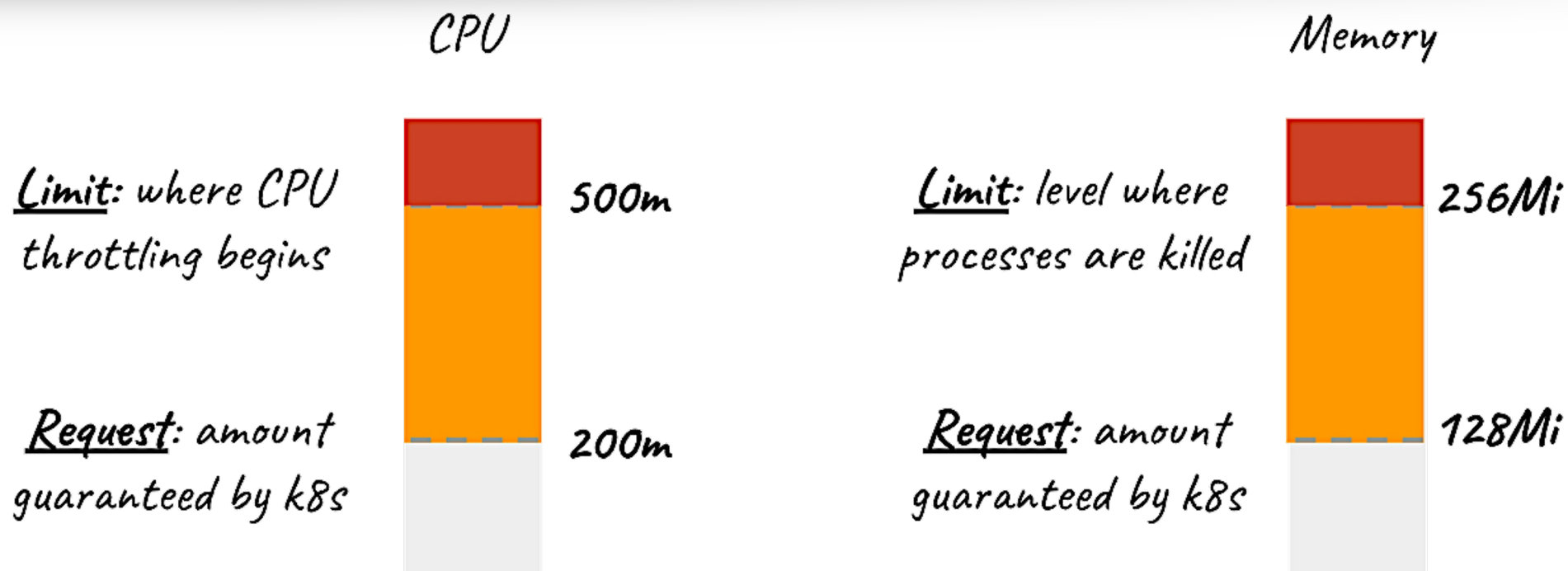
Reference <https://kubernetes.io/docs/api-reference/v1.6/>





► Pod详解





CPU 资源以 **CPU** 单位度量。Kubernetes 中的一个 CPU 等同于：

- 1 个 AWS vCPU
- 1 个 GCP 核心
- 1 个 Azure vCore
- 裸机上具有超线程能力的英特尔处理器上的 1 个超线程

小数值是可以使用的。一个请求 0.5 CPU 的容器保证会获得请求 1 个 CPU 的容器的 CPU 的一半。可以使用后缀 m 表示毫核。例如 100m CPU、100 milliCPU 和 0.1 CPU 都相同。精度不能超过 1m。CPU 请求只能使用绝对数量，而不是相对数量。0.1 在单核、双核或 48 核计算机上的 CPU 数量值是一样的。

- 标签与选择器结合使用来标识相关资源组。因为选择器是用来查询标签的，所以这个操作需要高效。为确保高效查询，标签受 RFC 1123[3] 约束。RFC 1123 以及其他约束将标签限制为最大 63 个字符长度。因此，当我们希望 Kubernetes 对一组相关资源进行分组时，应该使用标签。
- Annotations 用于非识别信息，即 Kubernetes 不关心的元数据。因此，注解键和值没有约束。因此，如果您想为其他人添加有关给定资源的信息，则注解是更好的选择。

简单来讲：Labels 适用于 Kubernetes，而 Annotations 适用于人类。

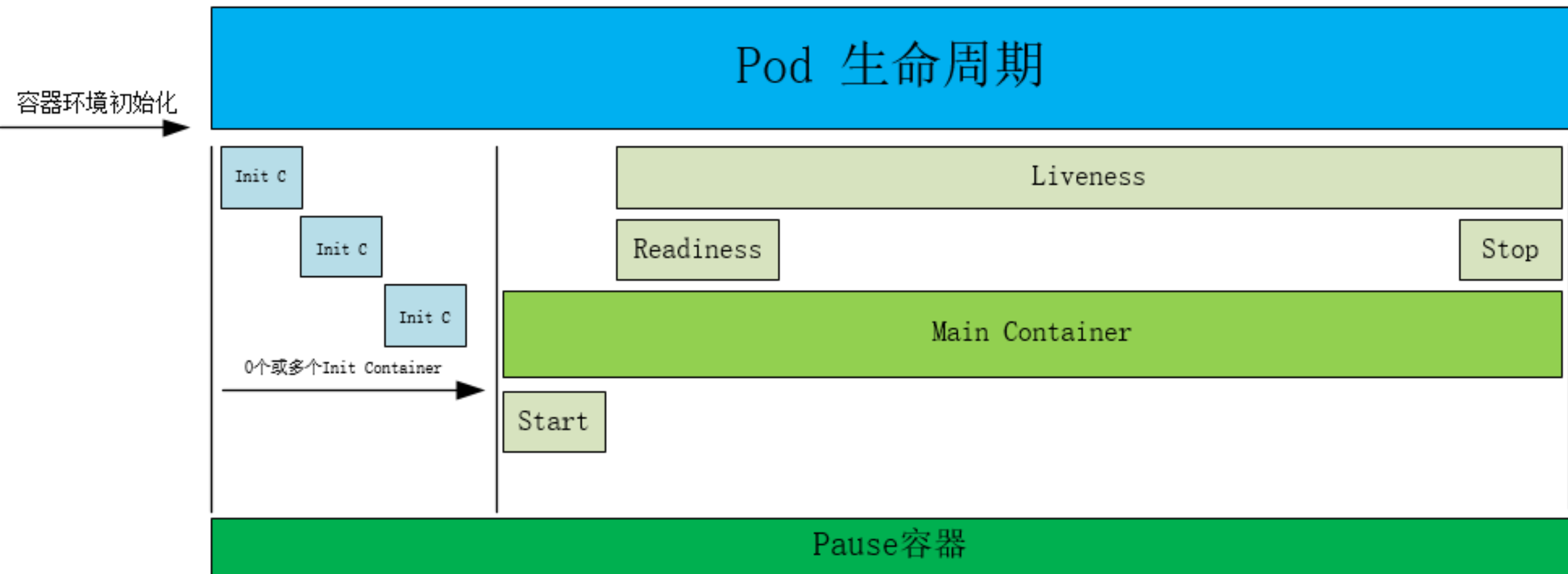
Label selector 是 Kubernetes 核心的分组机制，通过 label selector 客户端/用户能够识别一组有共同特征或属性的资源对象。

- 基于值相等的查询条件：类似于 SQL 语句中的 = 或 ! =
- 基于子集的查询条件：类似于 SQL 语句中的 in 或 not in
- environment in (production, qa)：所有键等于 environment 并且值等于 production 或者 qa 的资源
- tier notin (frontend, backend)：所有键等于 tier 并且值不等于 frontend 或者 backend 的资源，以及所有没有 tier 键标签的资源
- partition：所有包含了有 partition 标签的资源；没有校验它的值
- !partition：所有没有 partition 标签的资源；没有校验它的值
- 两种查询条件也可以组合在一起使用。

标签 是键值对。有效的标签键有两个段：可选的前缀和名称，用斜杠 (/) 分隔。名称段是必需的，必须小于等于 63 个字符，以字母数字字符 ([a-z0-9A-Z]) 开头和结尾，带有破折号 (-)，下划线 (_)，点 (.) 和之间的字母数字。前缀是可选的。如果指定，前缀必须是 DNS 子域：由点 (.) 分隔的一系列 DNS 标签，总共不超过 253 个字符，后跟斜杠 (/)。

如果省略前缀，则假定标签键对用户是私有的。向最终用户对象添加标签的自动系统组件（例如 kube-scheduler、kube-controller-manager、kube-apiserver、kubectl 或其他第三方自动化工具）必须指定前缀。

kubernetes.io/ 和 k8s.io/ 前缀是为 Kubernetes 核心组件保留的。



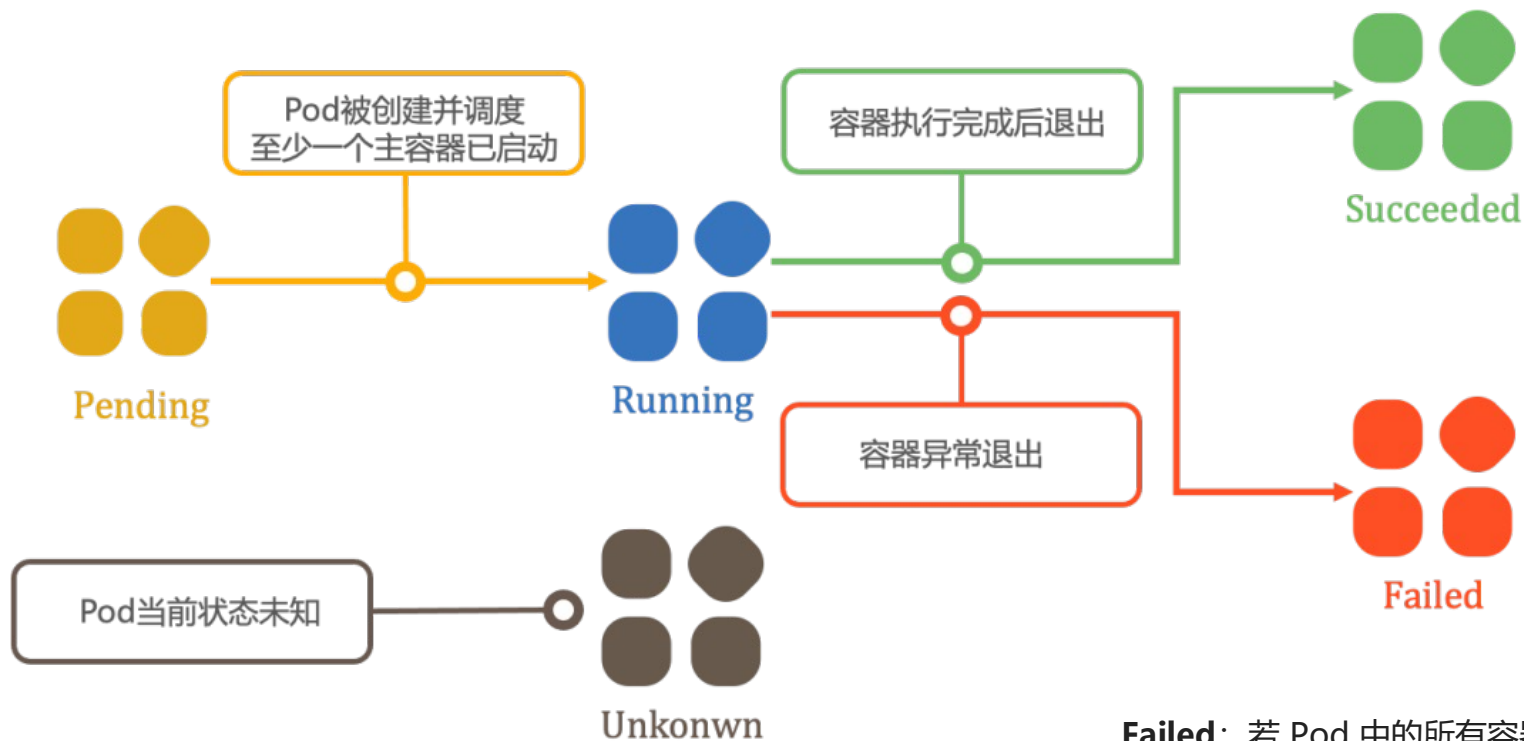
➤ 简要概述

- ✓ Init 容器是一种特殊容器，在 Pod 内的应用容器启动之前运行。Init 容器可以包括一些应用镜像中不存在的实用工具和安装脚本。
- ✓ 如果为一个 Pod 指定了多个 Init 容器，这些容器会按顺序逐个运行。每个 Init 容器必须运行成功，下一个才能够运行。当所有的 Init 容器运行完成时，Kubernetes 才会为 Pod 初始化应用容器并像平常一样运行。
- ✓ 使用 Init 容器可以：
 - Init 容器可以包含一些安装过程中应用容器中不存在的实用工具或个性化代码。例如，没有必要仅为了在安装过程中使用类似 sed、awk、python 或 dig 这样的工具而去 FROM 一个镜像来生成一个新的镜像。
 - Init 容器可以安全地运行这些工具，避免这些工具导致应用镜像的安全性降低。
 - 应用镜像的创建者和部署者可以各自独立工作，而没有必要联合构建一个单独的应用镜像。
 - Init 容器能以不同于 Pod 内应用容器的文件系统视图运行。因此，Init 容器可以访问应用容器不能访问的 Secret 的权限。
 - 由于 Init 容器必须在应用容器启动之前运行完成，因此 Init 容器提供了一种机制来阻塞或延迟应用容器的启动，直到满足了一组先决条件。一旦前置条件满足，Pod 内的所有的应用容器会并行启动。

Pending: Pod 被 K8s 创建出来后，起始于 Pending 阶段。在 Pending 阶段，Pod 将经过调度，被分配至目标节点开始拉取镜像、加载依赖项、创建容器。

Running: 当 Pod 所有容器都被创建，且至少一个容器已经在运行中，Pod 将进入 Running 阶段。

Succeeded: 当 Pod 中的所有容器都执行完成后终止，并且不会再重启，Pod 将进入 Succeeded 阶段。

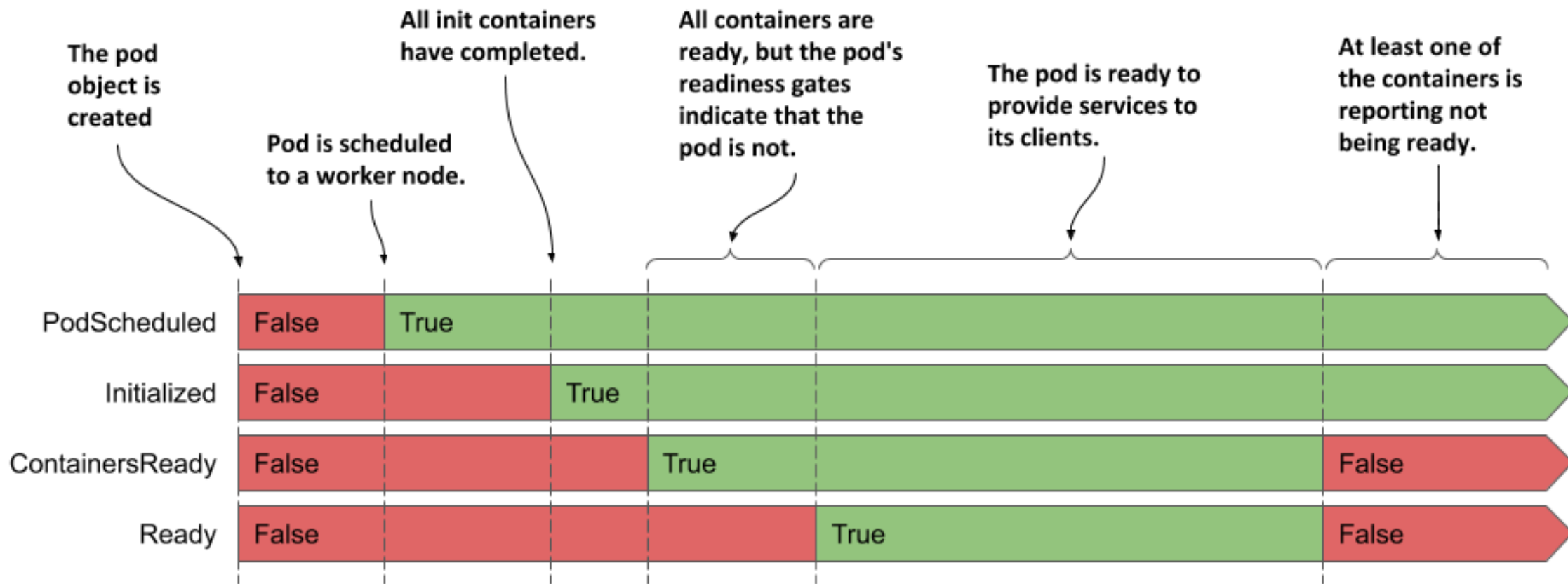


Unkonwn: 因为某些原因无法取得 Pod 状态，这种情况 Pod 将被置为 Unkonwn 状态。

Failed: 若 Pod 中的所有容器都已终止，并且至少有一个容器是因为失败终止，也就是说容器以非 0 状态异常退出或被系统终止，Pod 将进入 Failed 阶段。

Pod 的阶段 (Phase) 是 Pod 在其生命周期中所处位置的简单宏观概述。该阶段并不是对容器或 Pod 状态的综合汇总，也不是为了成为完整的状态机。

Pod 有一些细分状态（PodConditions），例如 Ready/NotReady、Initialized、PodScheduled/Unschedulable 等。这些细分状态描述造成 Pod 所处阶段的具体成因是什么。比如，Pod 当前阶段是 Pending，对应的细分状态是 Unschedulable，这就意味着 Pod 调度出现了问题。



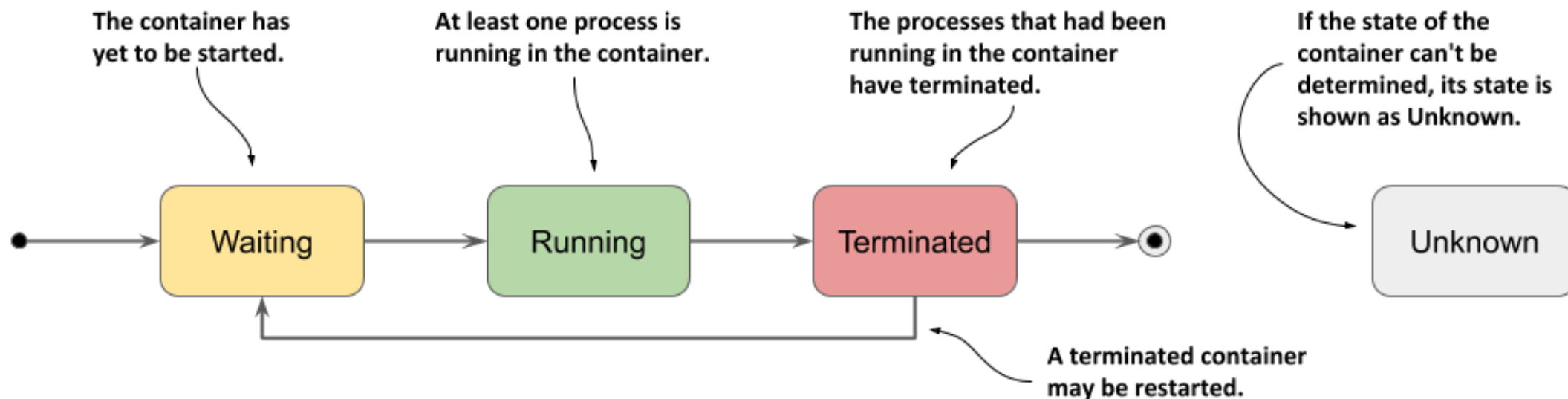
PodScheduled: Pod 已经被调度到某节点；

Initialized: 所有的 Init 容器都已成功完成；

ContainersReady: Pod 中所有容器都已就绪；

Ready: Pod 可以为请求提供服务，并且应该被添加到对应服务的负载均衡池中。

容器也有其生命周期状态 (State) : **Waiting**、**Running** 和 **Terminated**。并且也有其对应的状态原因 (Reason) , 例如 **ContainerCreating**、**Error**、**OOMKilled**、**ImagePullBackOff**、**CrashLoopBackOff**、**Completed** 等。而对于发生过重启或终止的容器, 上一个状态 (LastState) 字段不仅包含状态原因, 还包含上一次退出的状态码 (Exit Code) 。例如容器上一次退出状态码是 137, 状态原因是 OOMKilled, 说明容器是因为 OOM 被系统强行终止。在异常诊断过程中, 容器的退出状态是至关重要的信息。



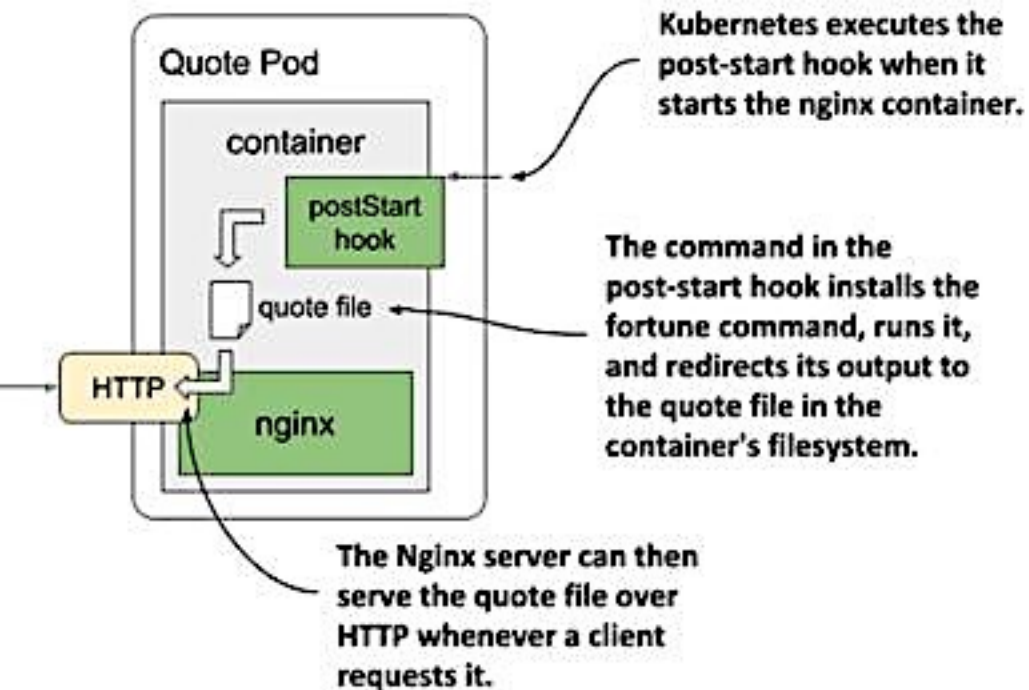
Kubernetes 会跟踪 Pod 中每个容器的状态, 我们可以使用[容器生命周期回调](#) 来在容器生命周期中的特定时间点触发事件。

有两个回调暴露给容器：

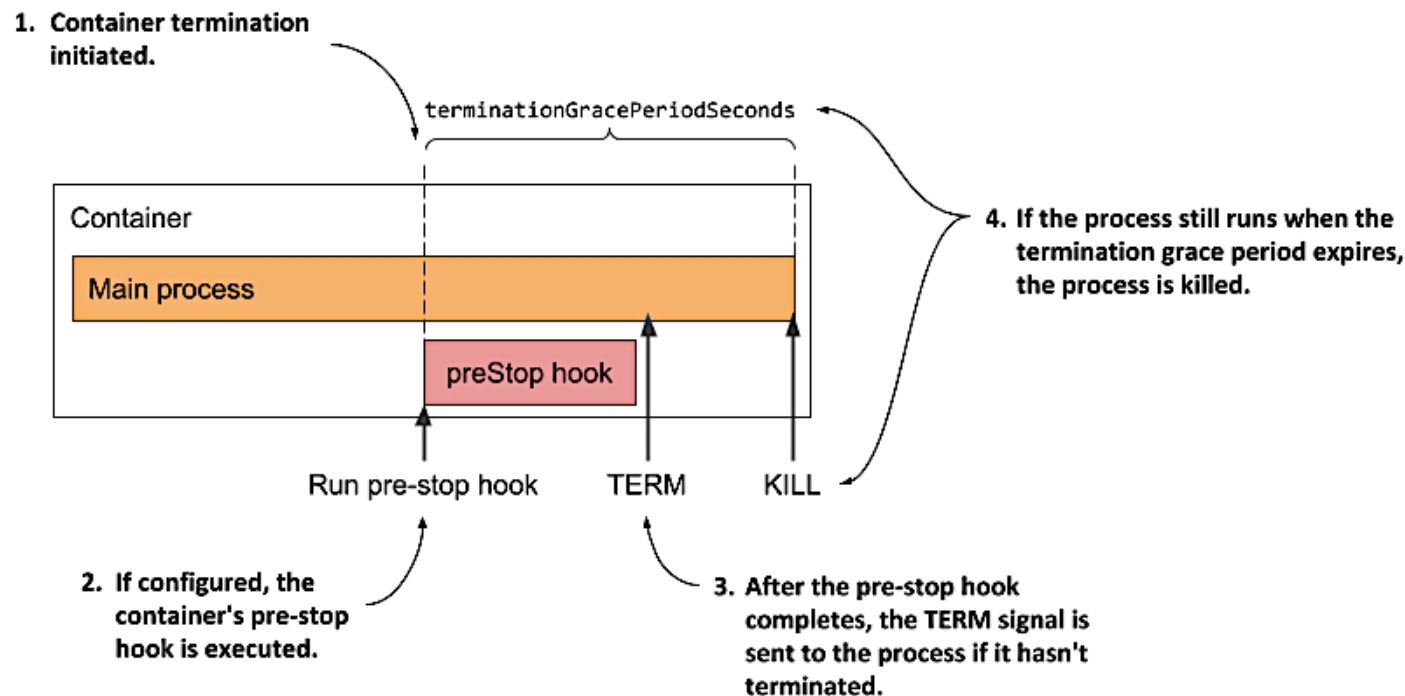
- **PostStart**: 这个回调在容器被创建之后立即被执行。但是不能保证回调会在容器入口点（ENTRYPOINT）之前执行。没有参数传递给处理程序。
- **PreStop**: 在容器因 API 请求或者管理事件（诸如存活态探针、启动探针失败、资源抢占、资源竞争等）而被终止之前，此回调会被调用。如果容器已经处于已终止或者已完成状态，则对 preStop 回调的调用将失败。在用来停止容器的 TERM 信号被发出之前，回调必须执行结束。Pod 的终止宽限周期在 PreStop 回调被执行之前即开始计数，所以无论回调函数的执行结果如何，容器最终都会在 Pod 的终止宽限期内被终止。没有参数会被传递给处理程序。

针对容器，有两种类型的回调处理程序可供实现：

- **Exec** - 在容器的 cgroups 和名称空间中执行特定的命令（例如 pre-stop.sh）。命令所消耗的资源计入容器的资源消耗。
- **HTTP** - 对容器上的特定端点执行 HTTP 请求。



postStart



PreStop

针对运行中的容器，kubelet 可以选择是否执行以下三种探针，以及如何针对探测结果作出反应：

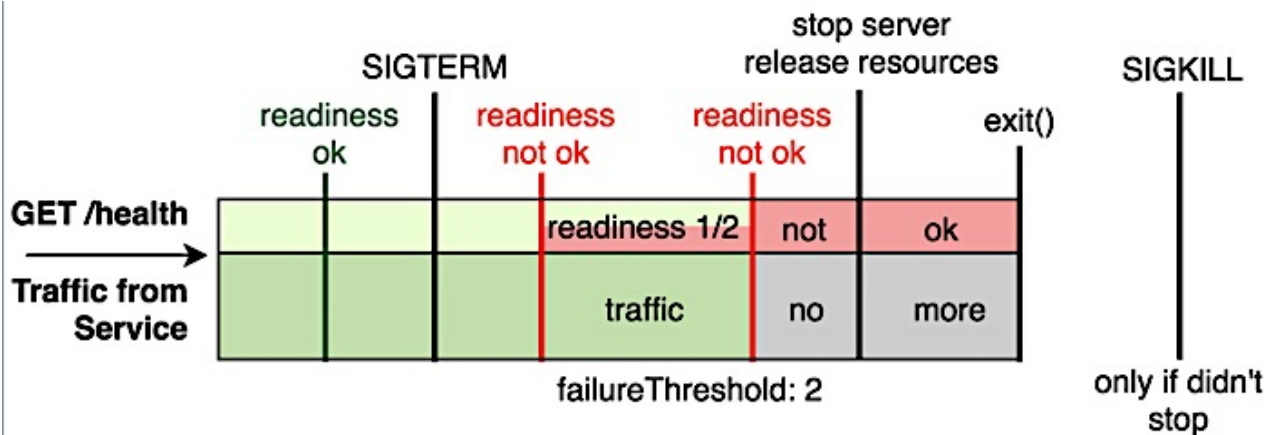
- **livenessProbe**：指示容器是否正在运行。如果存活态探测失败，则 kubelet 会杀死容器，并且容器将根据其重启策略决定未来。如果容器不提供存活探针，则默认状态为 Success。
- **readinessProbe**：指示容器是否准备好为请求提供服务。如果就绪态探测失败，端点控制器将从与 Pod 匹配的所有服务的端点列表中删除该 Pod 的 IP 地址。初始延迟之前的就绪态的状态值默认为 Failure。如果容器不提供就绪态探针，则默认状态为 Success。
- **startupProbe**：指示容器中的应用是否已经启动。如果提供了启动探针，则所有其他探针都会被禁用，直到此探针成功为止。如果启动探测失败，kubelet 将杀死容器，而容器依其重启策略进行重启。如果容器没有提供启动探测，则默认状态为 Success。

探针支持三种类型的处理程序：

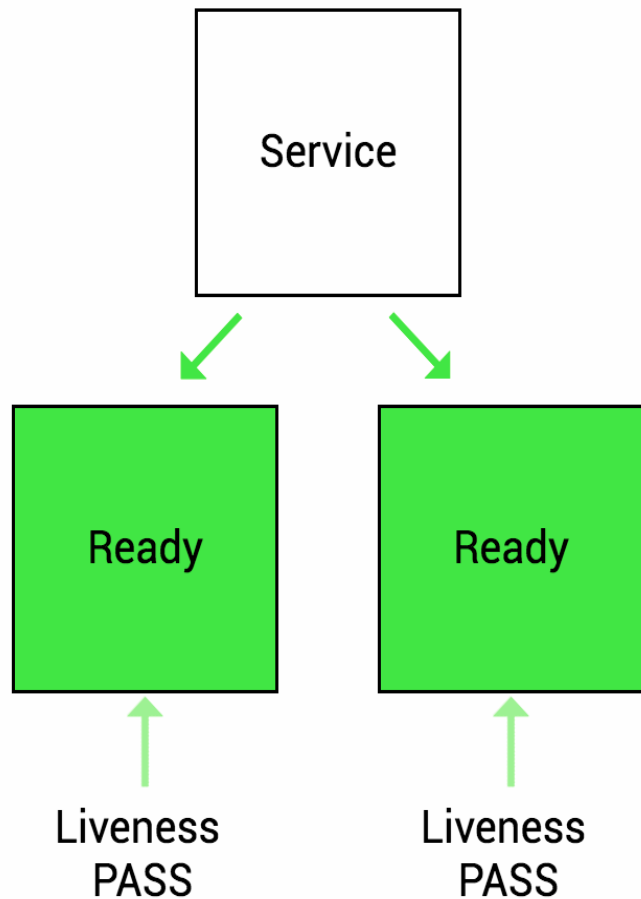
- **ExecAction**：在容器内执行指定命令。如果命令退出时返回码为 0 则认为诊断成功。
- **TCPSocketAction**：对容器的 IP 地址上的指定端口执行 TCP 检查。如果端口打开，则诊断被认为是成功的。
- **HTTPGetAction**：对容器的 IP 地址上指定端口和路径执行 HTTP Get 请求。如果响应的状态码大于等于 200 且小于 400，则诊断被认为是成功的。

每次探测都将获得以下三种结果之一：

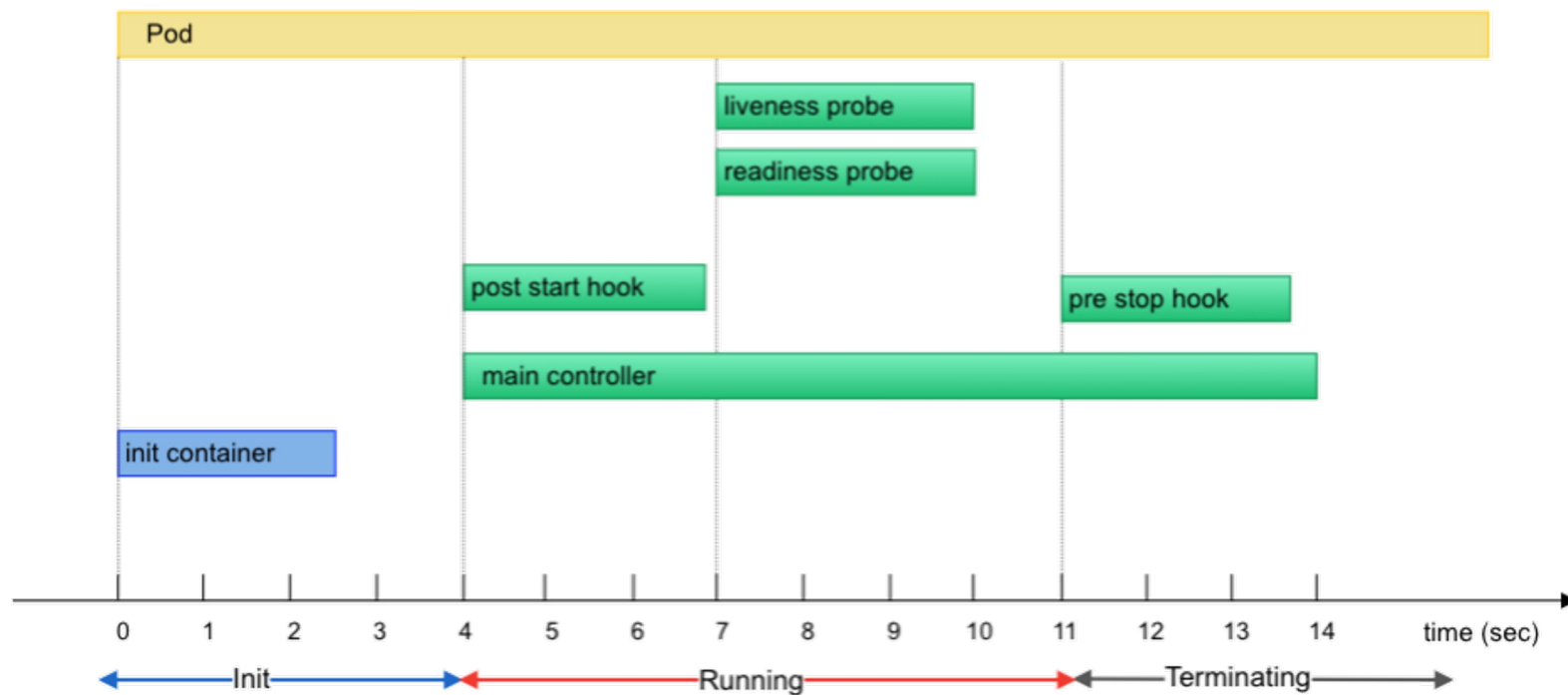
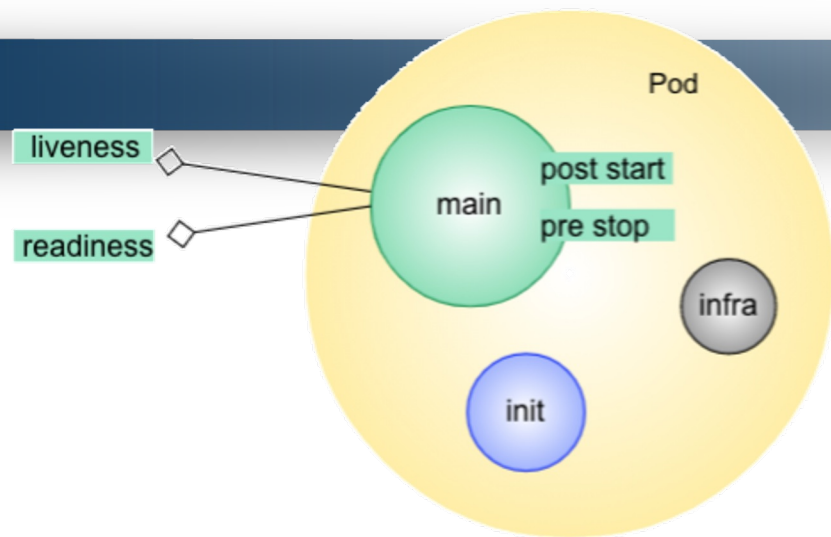
- **Success**（成功）：容器通过了诊断。
- **Failure**（失败）：容器未通过诊断。
- **Unknown**（未知）：诊断失败，因此不会采取任何行动。



Readiness 检测用于是否允许流量接入



Liveness 检查用于判断容器是否健康，告诉 kubelet 一个容器什么时候处于不健康的状态 – 重启 Pod



Pod lifecycle



2 chapter

K8S设计理念

- ✓ 基本概念回顾
- ✓ 设计理念

➤ 简要概述

- Cluster: 集群是指由Kubernetes使用一系列的物理机、虚拟机和其他基础资源来运行你的应用程序
- Node: 一个Node就是一个运行着Kubernetes的物理机或虚拟机，并且pod可以在其上面被调度
- Pod: 一个Pod对应一个由相关容器和卷组成的容器组（了解Pod详情）
- Label: 一个被附加到资源上的键/值对，譬如附加到一个Pod上，为它传递一个用户自定义的并且可识别的属性；还可以被应用来组织和选择子网中的资源（了解Label详情）
- Selector: 一个通过匹配labels来定义资源之间关系得表达式，例如为一个负载均衡的service指定目标Pod。（了解selector详情）

➤ 简要概述

- **Replication Controller** : replication controller 是为了保证一定数量被指定的Pod的复制品在任何时间都能正常工作.它不仅允许复制的系统易于扩展,还会处理当pod在机器在重启或发生故障的时候再次创建一个 (了解Replication Controller详情)
- **Service** : 一个 service 定义了访问 pod 的方式,就像单个固定的IP地址和与其相对应的DNS名之间的关系。 (了解Service详情)
- **Volume**: 一个 **volume** 是一个目录,可能会被容器作为文件系统的一部分来访问。 (了解Volume详情)。Kubernetes volume 构建在 Docker Volumes 之上,并且支持添加和配置 volume 目录或者其他存储设备。

➤ 简要概述

- **Secret** : Secret 存储了敏感数据, 例如能允许容器接收请求的权限令牌。
- **Name** : 用户为 Kubernetes 中资源定义的名字
- **Namespace** : Namespace 好比一个资源名字的前缀。它帮助不同的项目、团队或是客户可以共享 cluster, 例如防止相互独立的团队间出现命名冲突
- **Annotation** : 相对于 label 来说可以容纳更大的键值对, 它对我们来说可能是不可读的数据, 只是为了存储不可识别的辅助数据, 尤其是一些被工具或系统扩展用来操作的数据

➤ 分层架构



➤ 系统API设计原则

- 对于云计算系统，系统API实际上处于系统设计的统领地位
- K8s 集群系统每支持一项新功能，引入一项新技术，一定会新引入对应的 API 对象，支持对该功能的管理操作，理解掌握的 API，就好比抓住了 k8s 系统的牛鼻子
- K8s 系统 API 的设计有以下几条原则：
 - 所有 API 应该是声明式的
 - API 对象是彼此互补并且可以组合的
 - 高层 API 以操作意图为基础设计
 - 底层 API 根据高层 API 的控制需要设计。
 - 尽量避免简单封装，不要有外部 API 无法显式知道的内部隐藏的机制
 - API 操作复杂度与对象数量成正比。
 - API 对象状态不能依赖于网络连接状态。
 - 尽量避免让操作机制依赖于全局状态。

➤ 控制机制设计原则

- 控制逻辑应该只依赖于当前状态。
- 假设任何错误的可能，并做容错处理。
- 尽量避免复杂状态机，控制逻辑不要依赖无法监控的内部状态。
- 假设任何操作都可能被任何操作对象拒绝，甚至被错误解析。
- 每个模块都可以在出错后自动恢复。
- 每个模块都可以在必要时优雅地降级服务。（所谓优雅地降级服务，是对系统鲁棒性的要求）



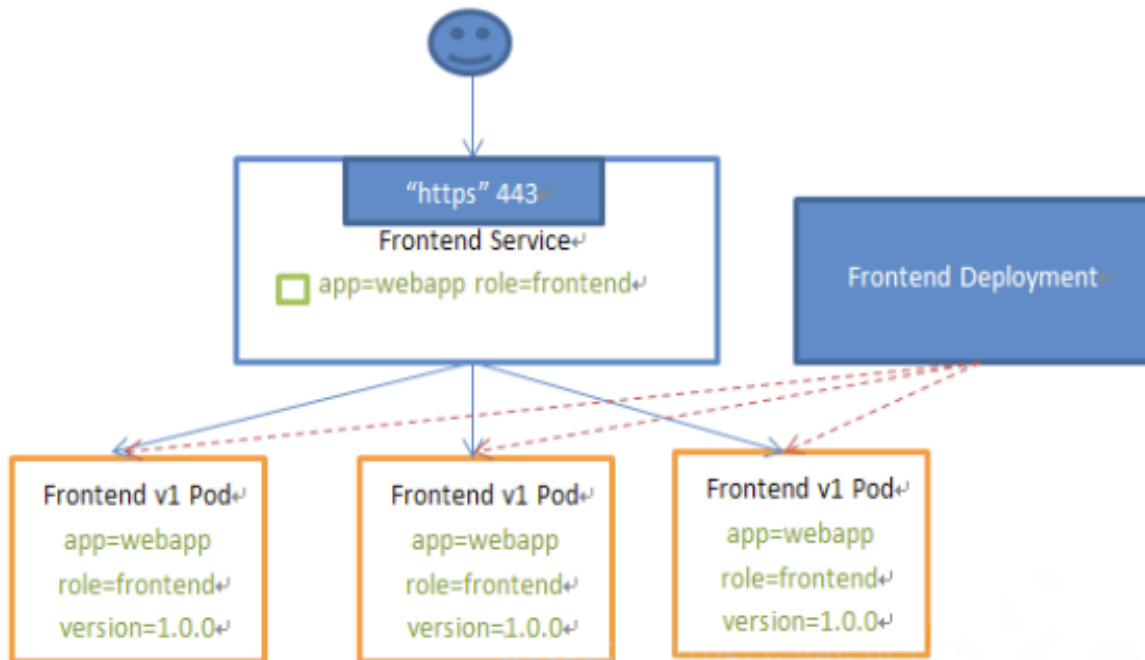
3 chapter

K8S工作机制

- ✓ 核心资源对象详解
- ✓ Kubernetes 网络
- ✓ Kubernetes 存储
- ✓ Kubernetes 安全
- ✓ Kubernetes 集群调度

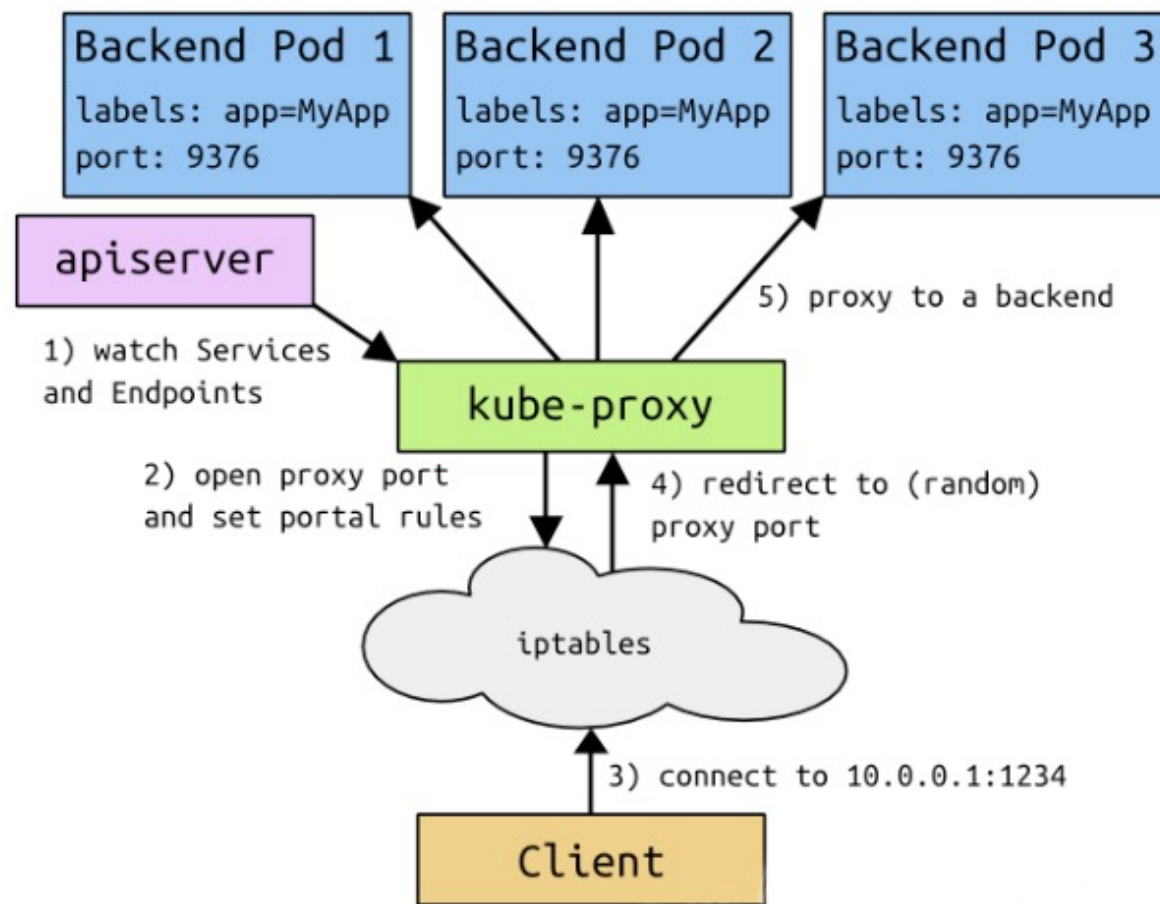
➤ SVC 详解

- Kubernetes Service 定义了这样一种抽象：逻辑上的一组 Pod，一种可以访问它们的策略 —— 通常称为微服务。Service 所针对的 Pods 集合通常是通过选择算符来确定的。
 - 举个例子，考虑一个图片处理后端，它运行了 3 个副本。这些副本是可互换的 —— 前端不需要关心它们调用了哪个后端副本。然而组成这一组后端程序的 Pod 实际上可能会发生变化，前端客户端不应该也没必要知道，而且也不需要跟踪这一组后端的状态。



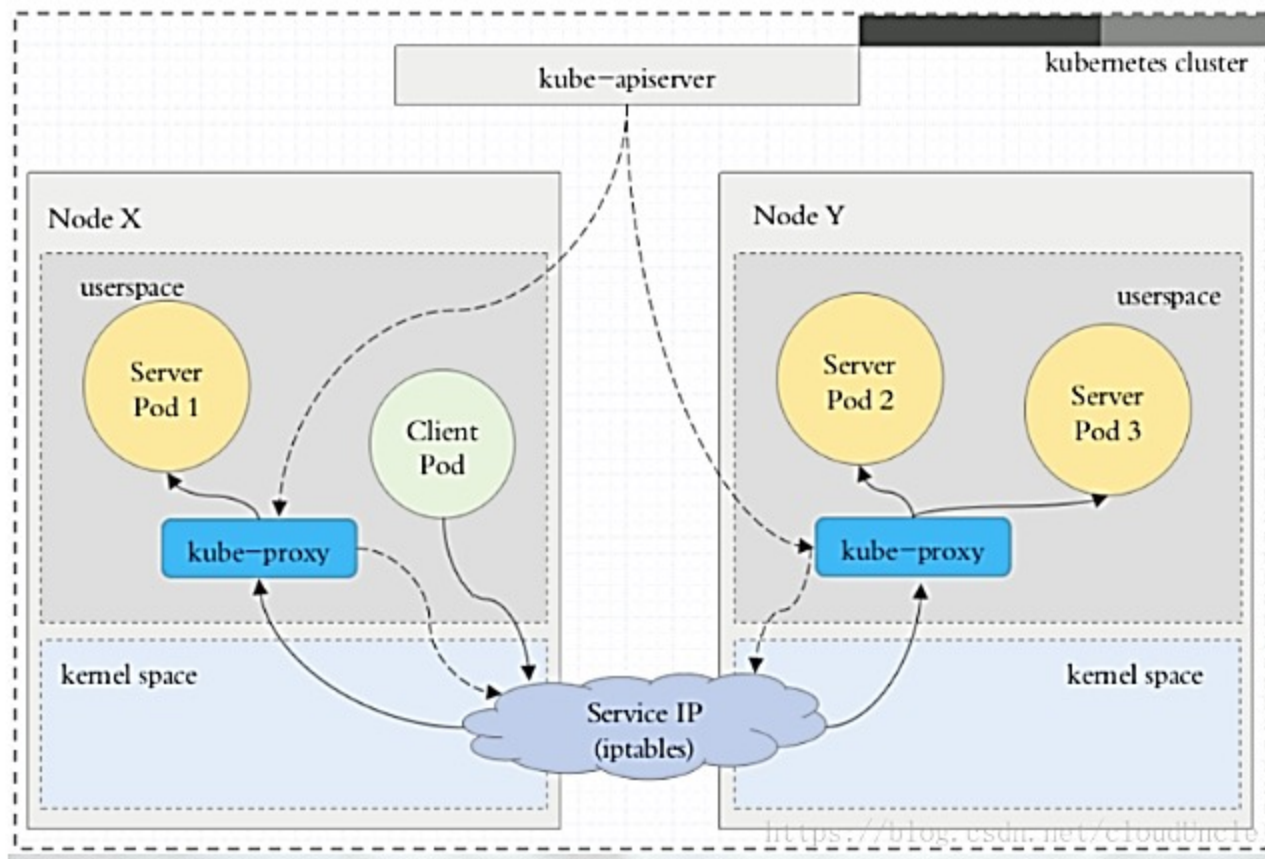
➤ SVC 详解

- Service在K8s中有以下四种类型
 - ClusterIp、NodePort、LoadBalancer、ExternalName



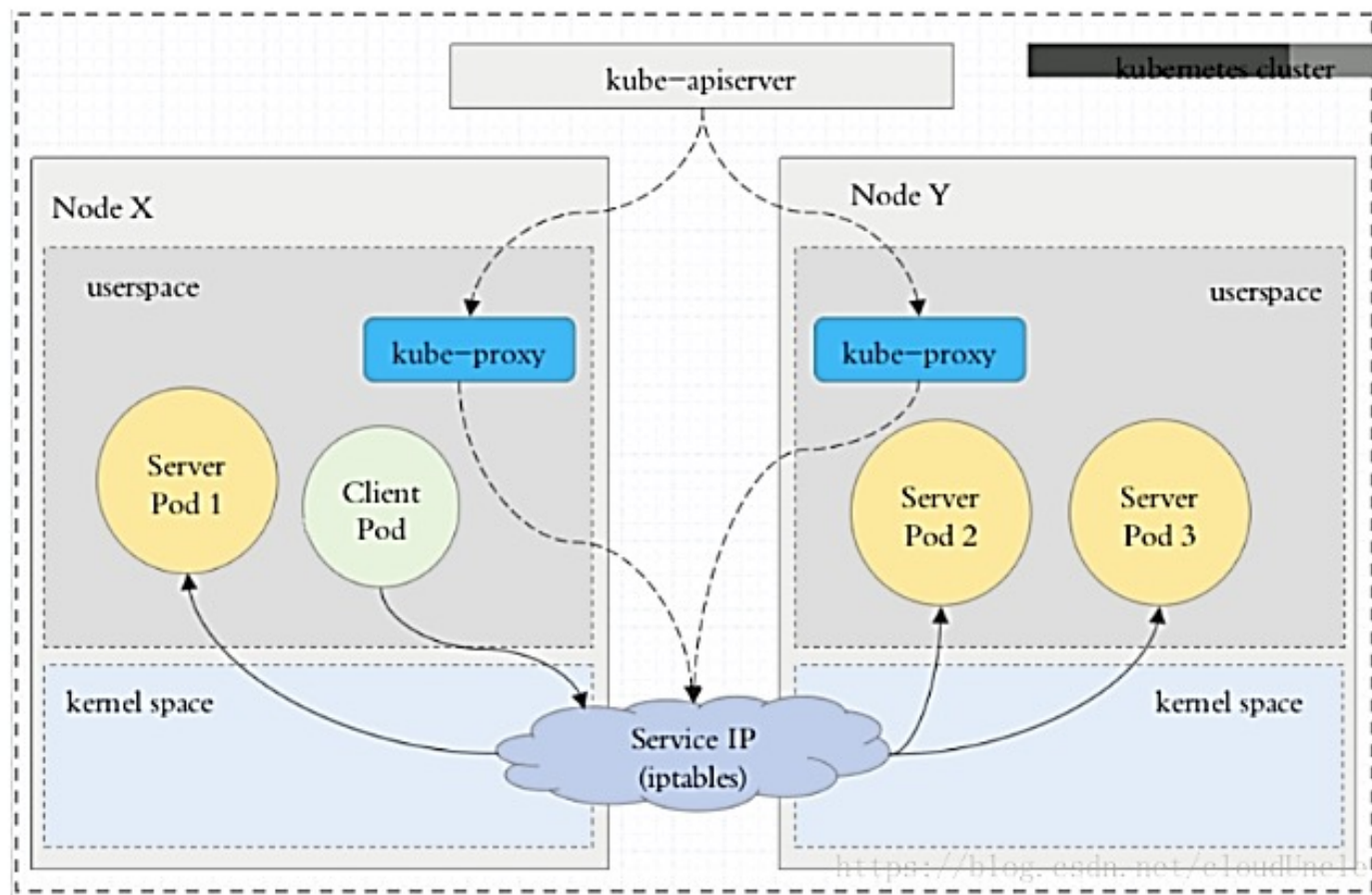
➤ SVC 详解

- 代理模式1: userspace



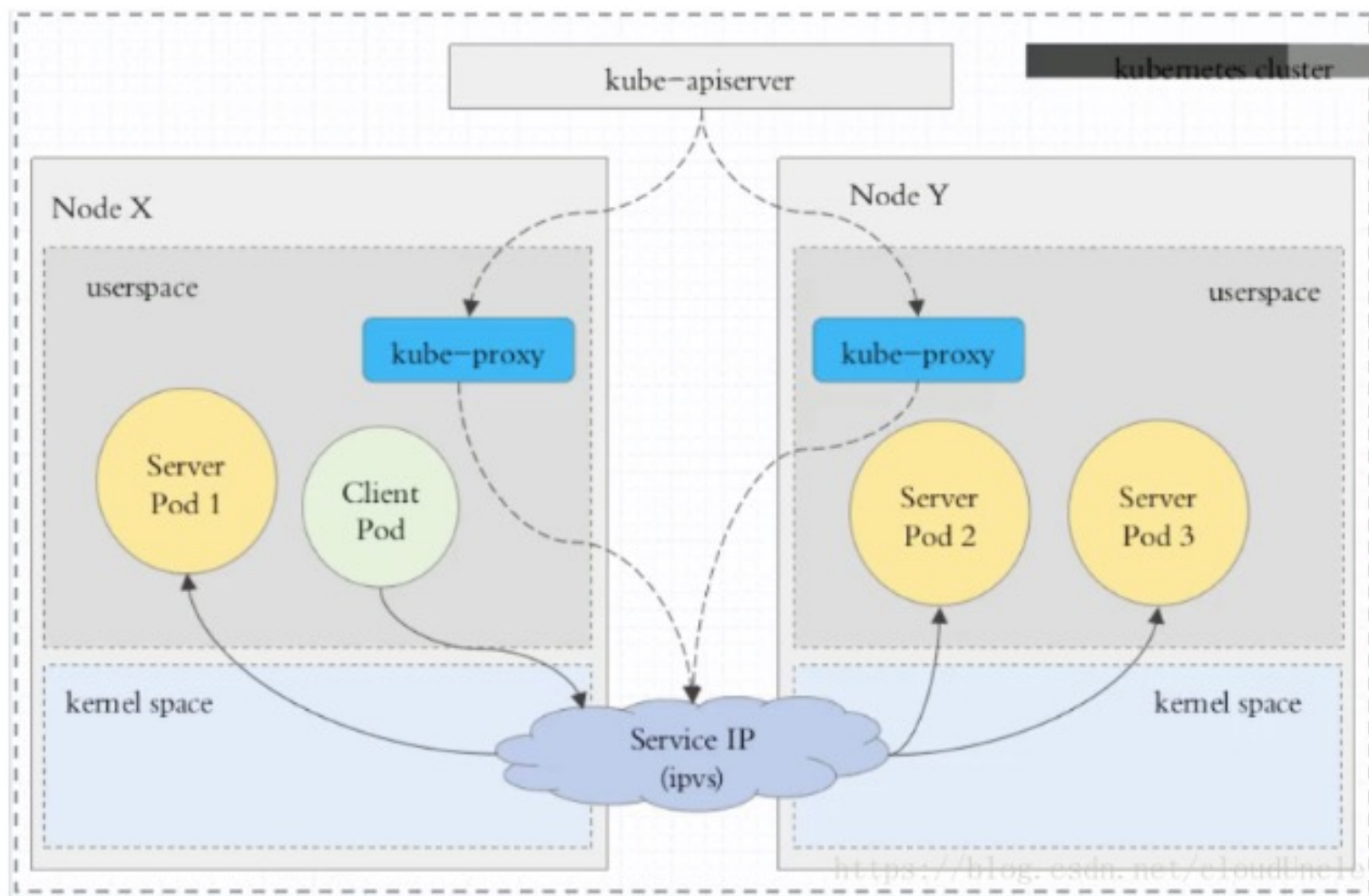
➤ SVC 详解

- 代理模式2: iptables



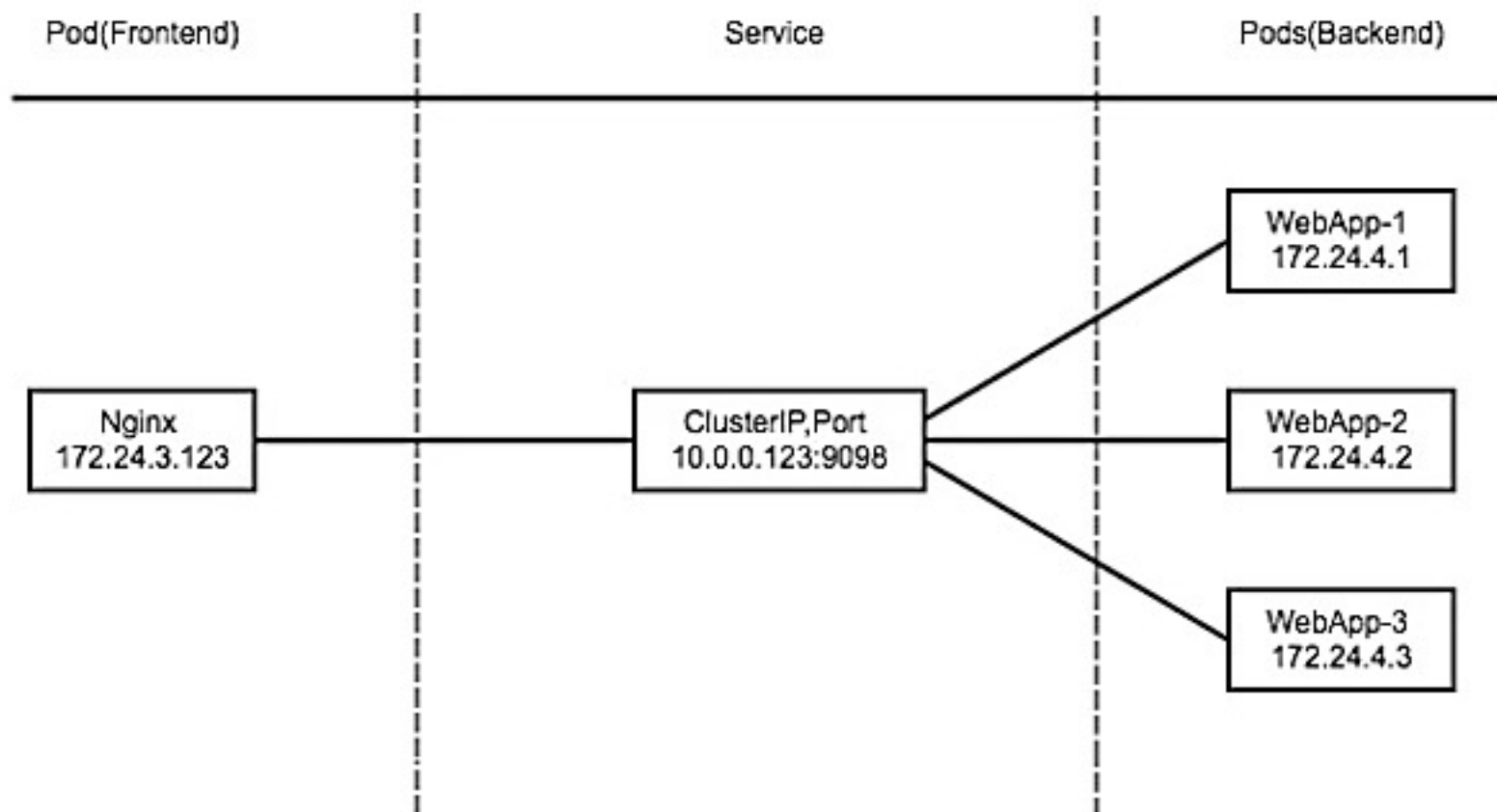
➤ SVC 详解

- 代理模式3: ipvs



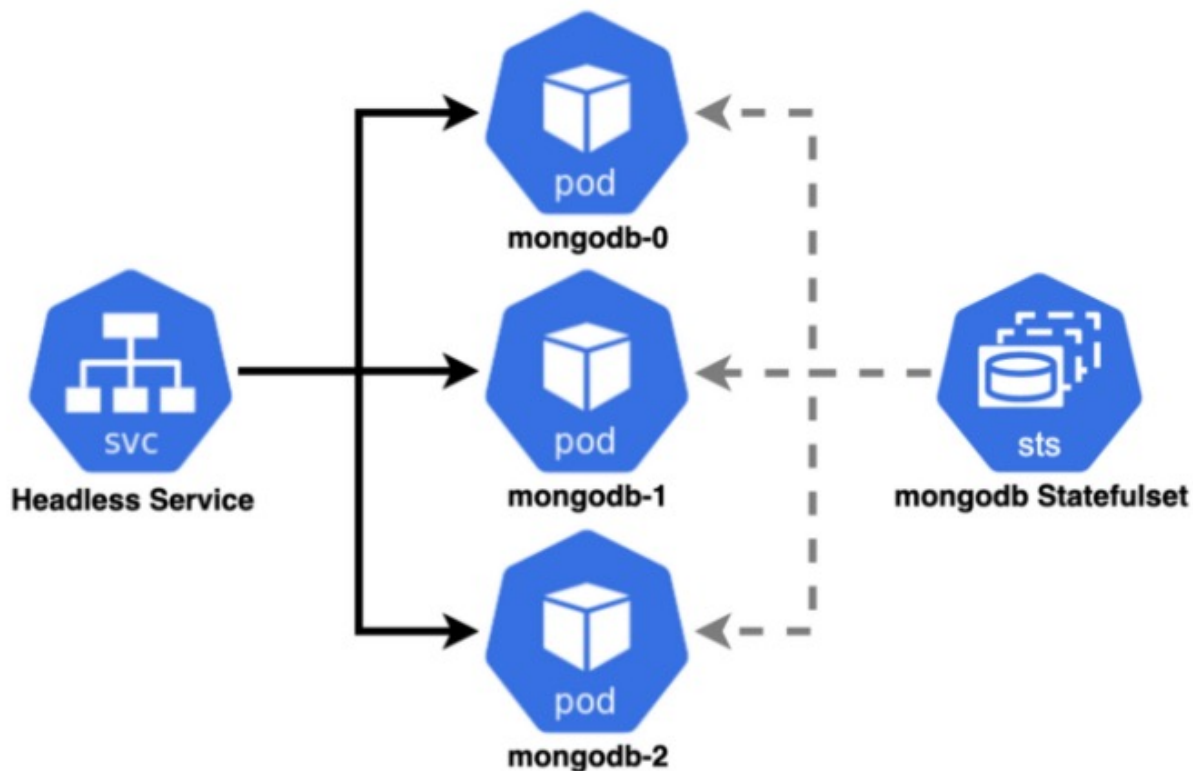
➤ SVC 详解

- clusterIP: 主要是在每个node节点使用iptables, 将发向clusterIP对应端口的数据, 转发到kube-proxy中。



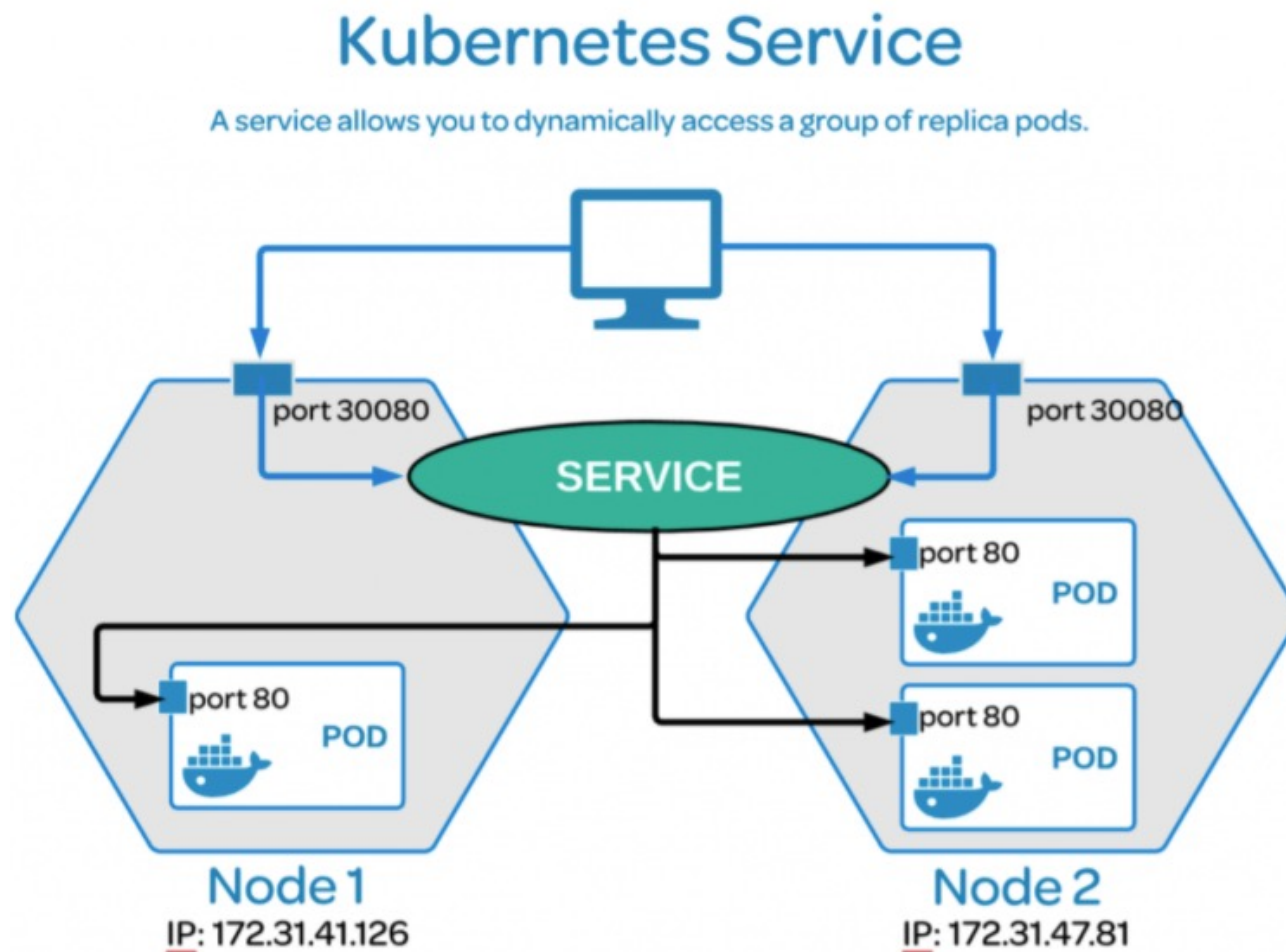
➤ SVC 详解

- **Headless Service:** 有时不需要或不想要负载均衡,以及单独的 Service IP。遇到这种情况,可以通过指定Cluster IP(spec.clusterIP)的值为“None”来创建 Headless Service。这类 Service 并不会分配 Cluster IP, kube-proxy 不会处理它们,而且平台也不会为它们进行负载均衡和路由。



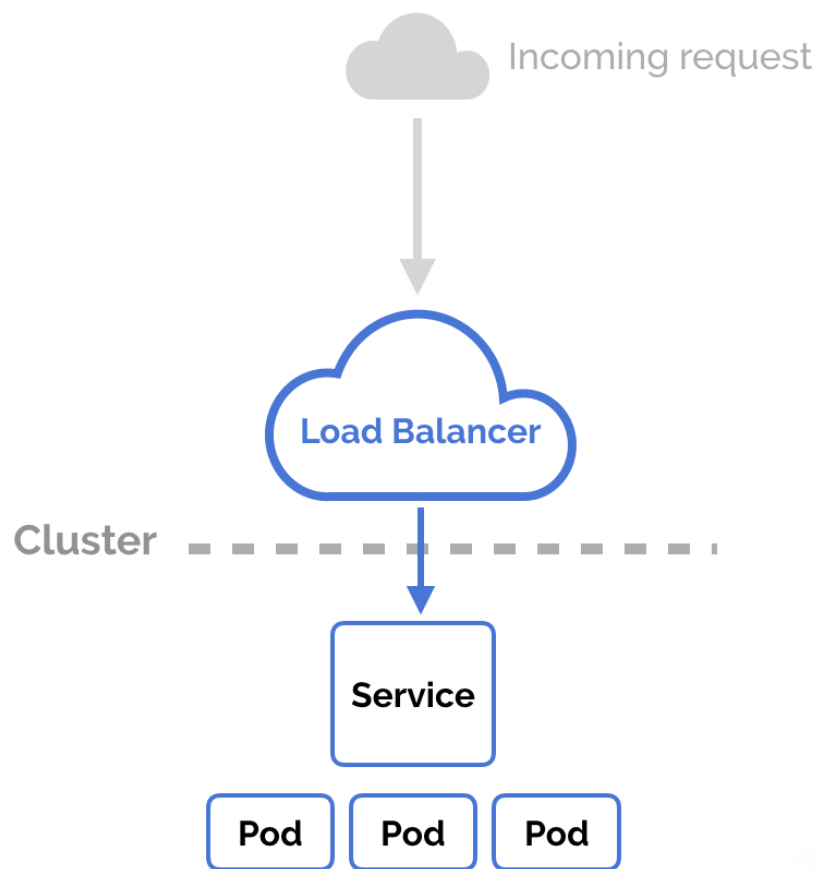
➤ SVC 详解

- nodePort: 的原理在于在 node 上开了一个端口，将向该端口的流量导入到 kube-proxy，然后由 kube-proxy 进一步到给对应的 pod。

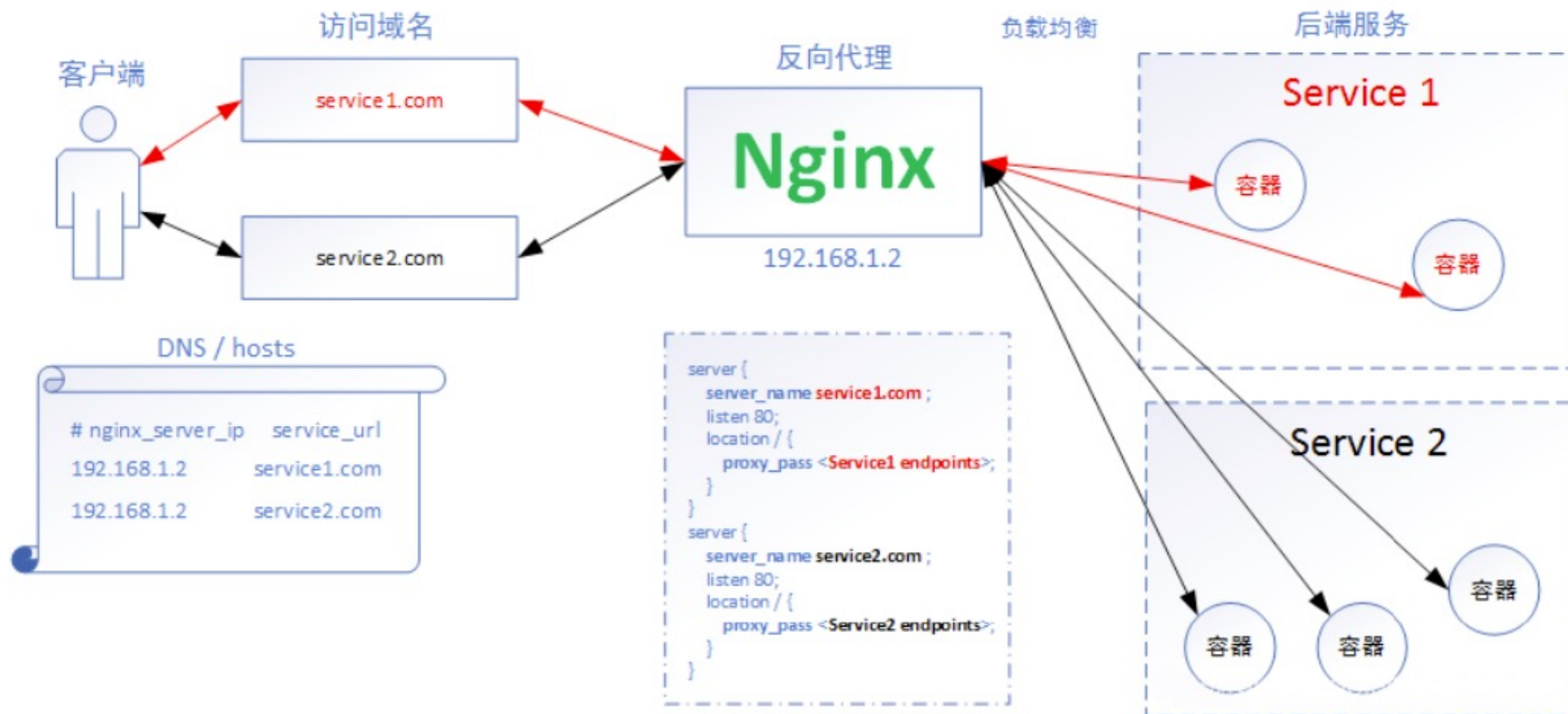


➤ SVC 详解 - LoadBalancer

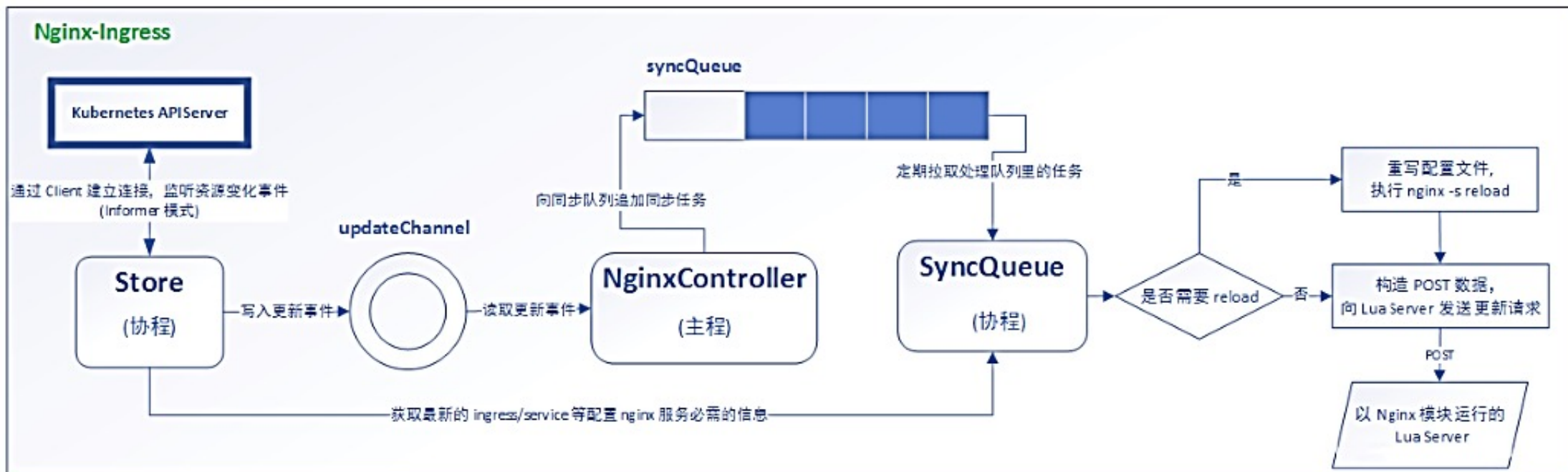
- loadBalancer 和 nodePort 其实是同一种方式。区别在于 loadBalancer 比 nodePort 多了一步，就是可以调用 cloud provider 去创建 LB 来向节点导流。



➤ SVC 详解 - 扩展：Ingress-Nginx



➤ SVC 详解 - 扩展：Ingress-Nginx

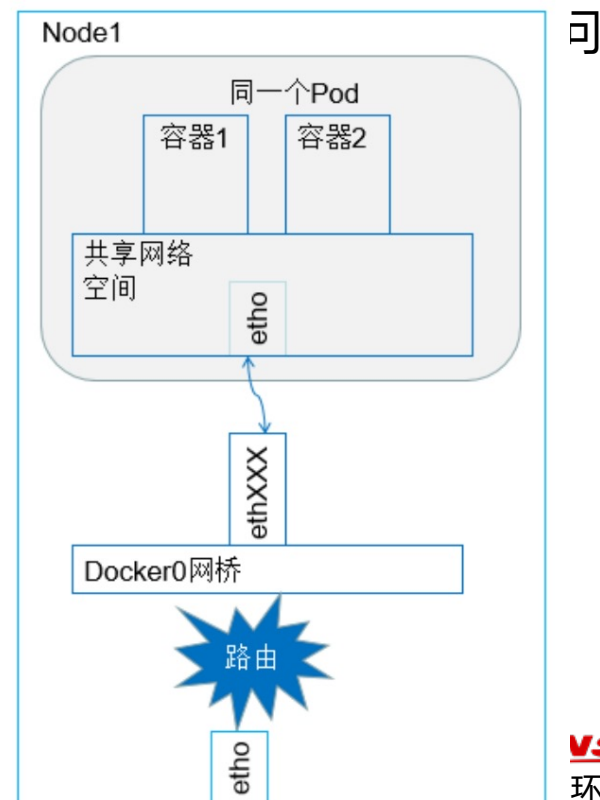


➤ 基础原则

- 每个 Pod 都拥有一个独立的 IP 地址，并假定所有 Pod 都在一个可以直接连通的、扁平的网络空间中。
- 不管它们是否运行在同一个 Node（宿主机）中，都要求它们可以直接通过对方的 IP 进行访问。
- 用户不需要额外考虑如何建立 Pod 之间的连接，也不需要考虑如何将

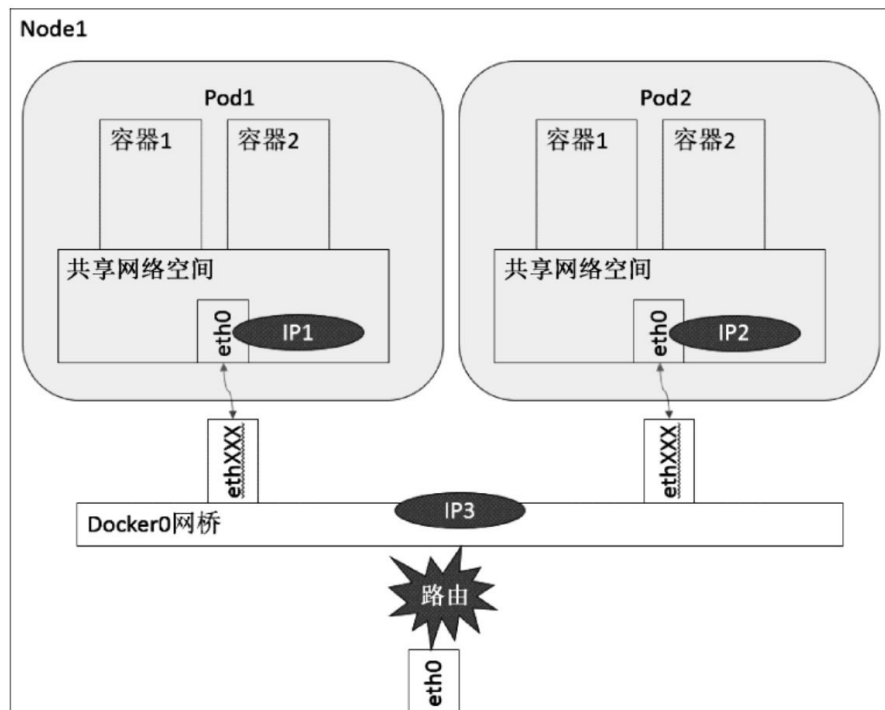
➤ 具体实现

- 实际上，在 Kubernetes 的世界里，IP 是以 Pod 为单位进行分配的。
- 一个 Pod 内部的所有容器共享一个网络堆栈
- 按照这个网络原则抽象出来的为每个 Pod 都设置一个 IP 地址的模型也被称作 IP-per-Pod 模型。



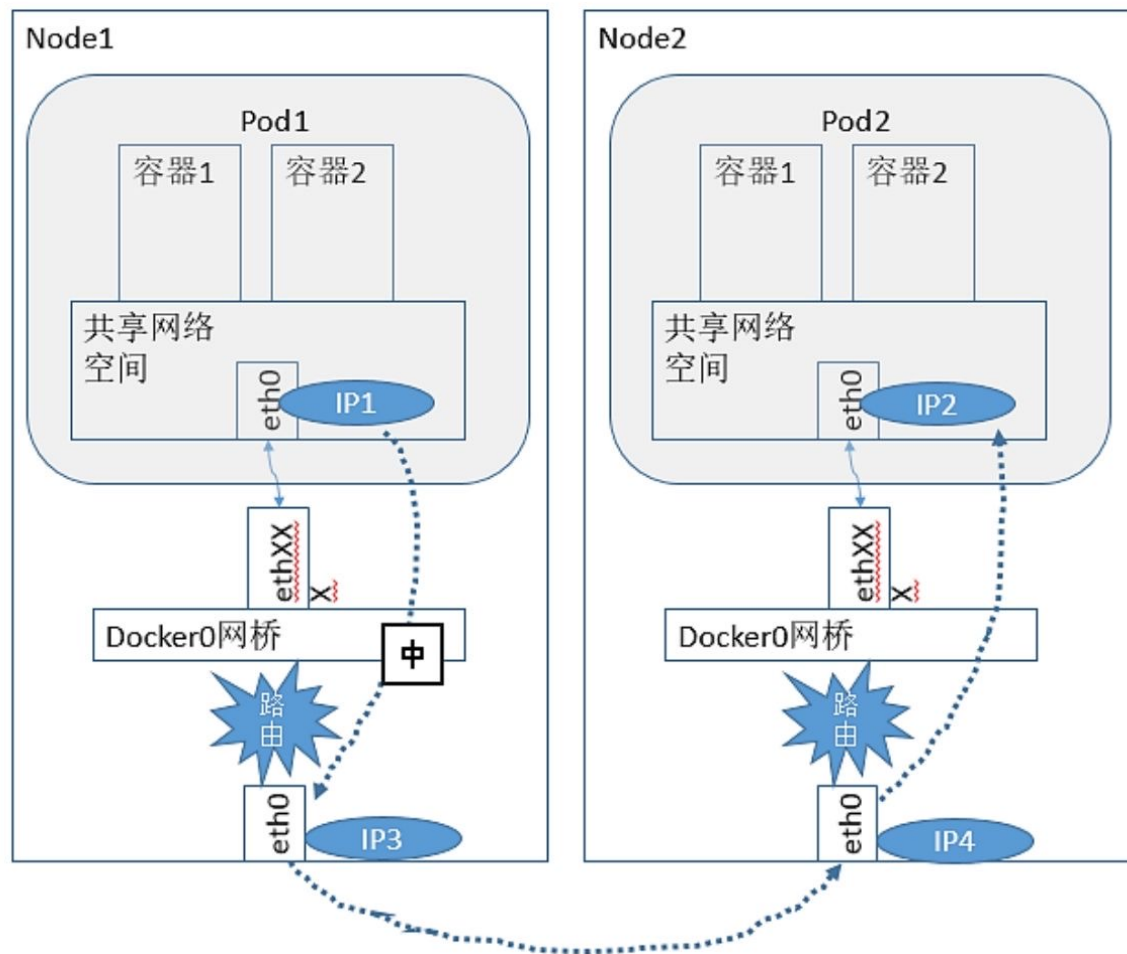
➤ Pod之间的通信

- 每一个 Pod 都有一个真实的全局 IP 地址，同一个 Node 内的不同 Pod 之间可以直接采用对方 Pod 的 IP 地址通信，而且不需要采用其他发现机制，例如 DNS、Consul 或者 etcd。
- Pod 容器既有可能在同一个 Node 上运行，也有可能在不同的 Node 上运行，所以通信也分为两类：同一个 Node 内 Pod 之间的通信和不同 Node 上 Pod 之间的通信。
- 同一个 Node 内 Pod 之间的通信：

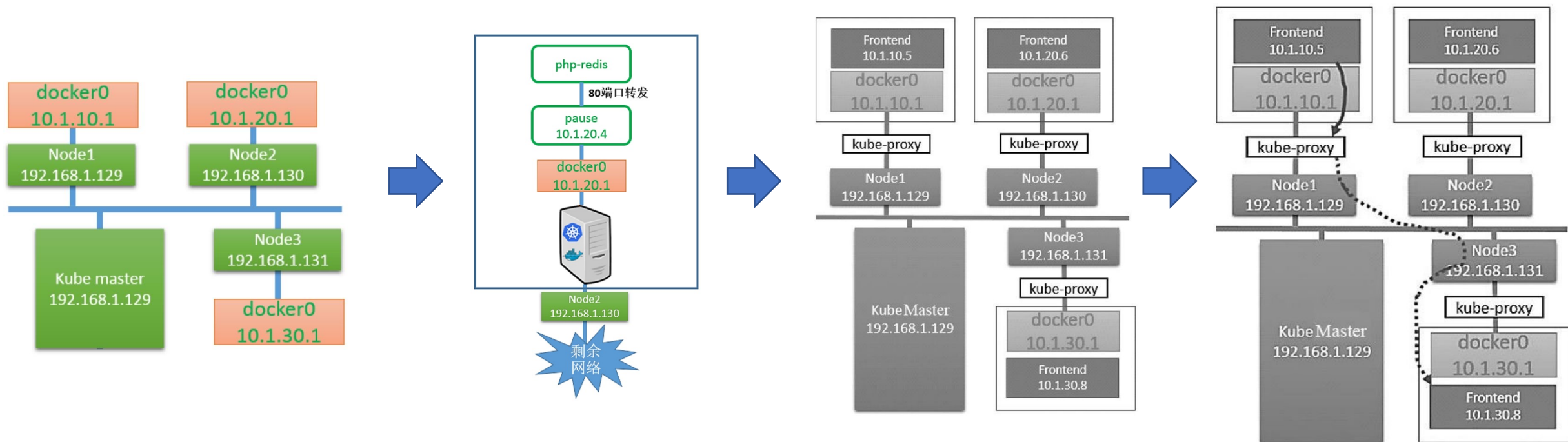


➤ Pod之间的通信

- 不同 Node 上 Pod 之间的通信

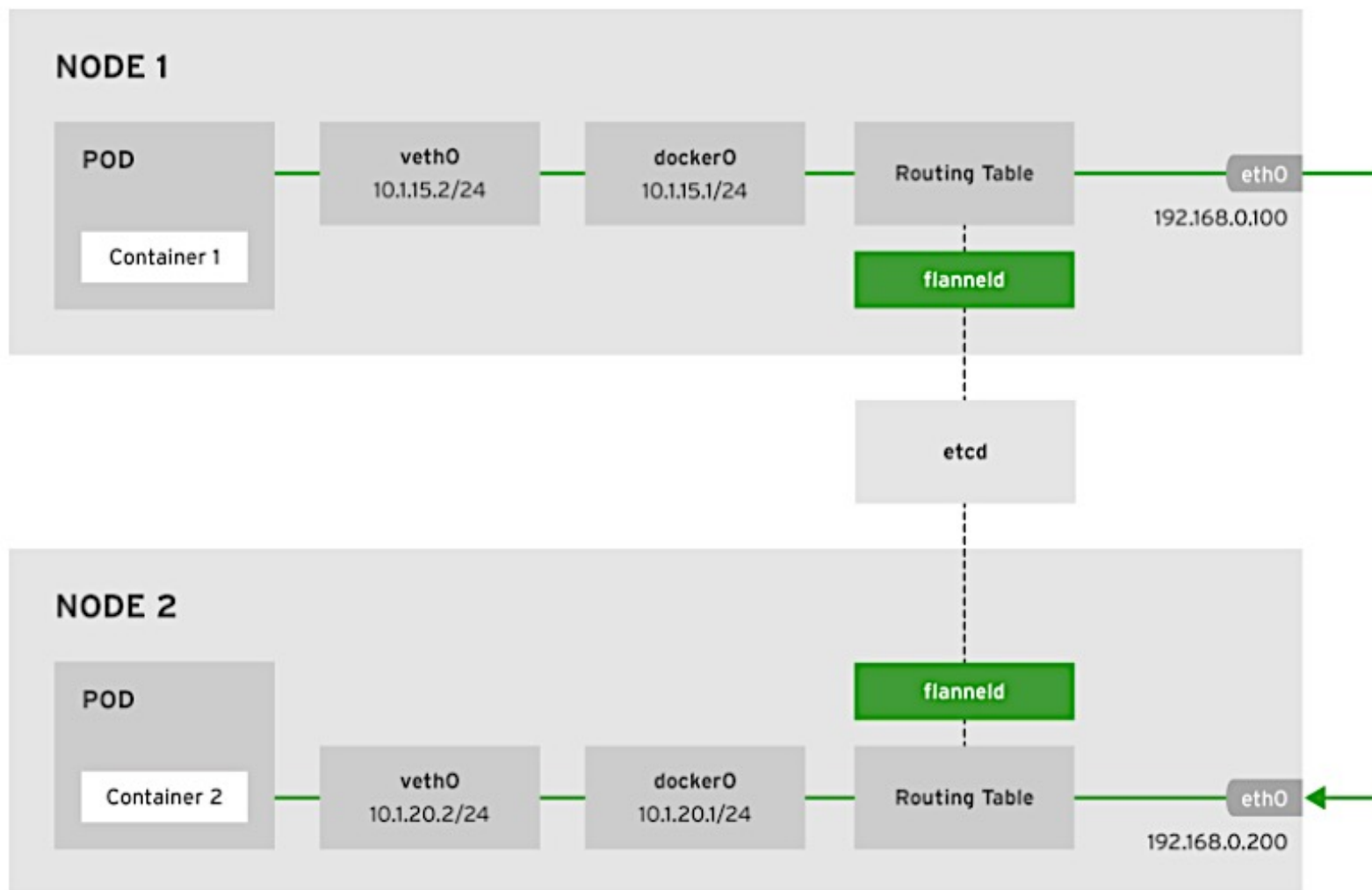


➤ SVC 到 Pod 的通信

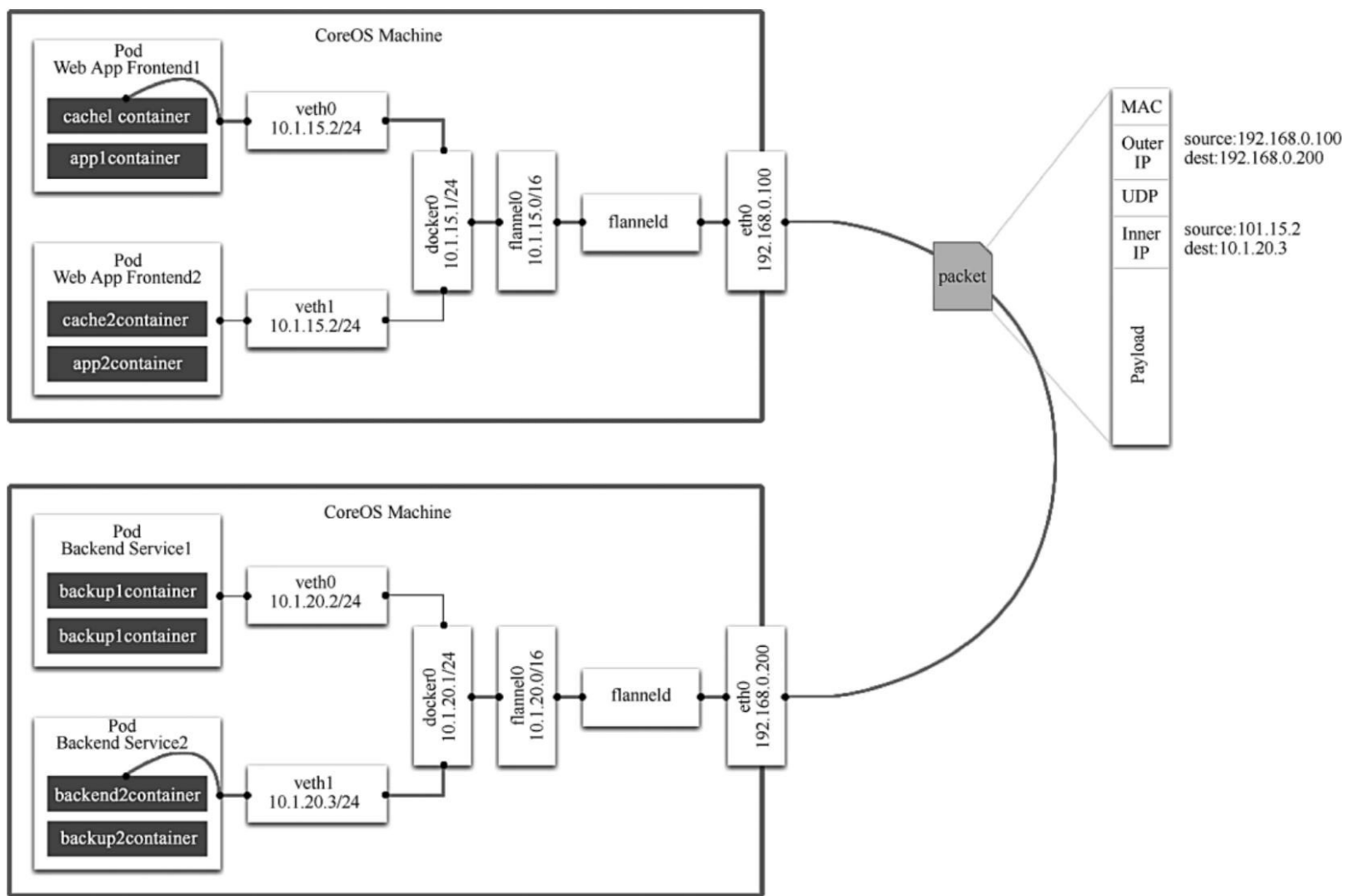


➤ Flannel (常用解决方案)

- Kubernetes 本身并不提供网络功能，只是把网络接口开放出来，通过插件的形式实现。

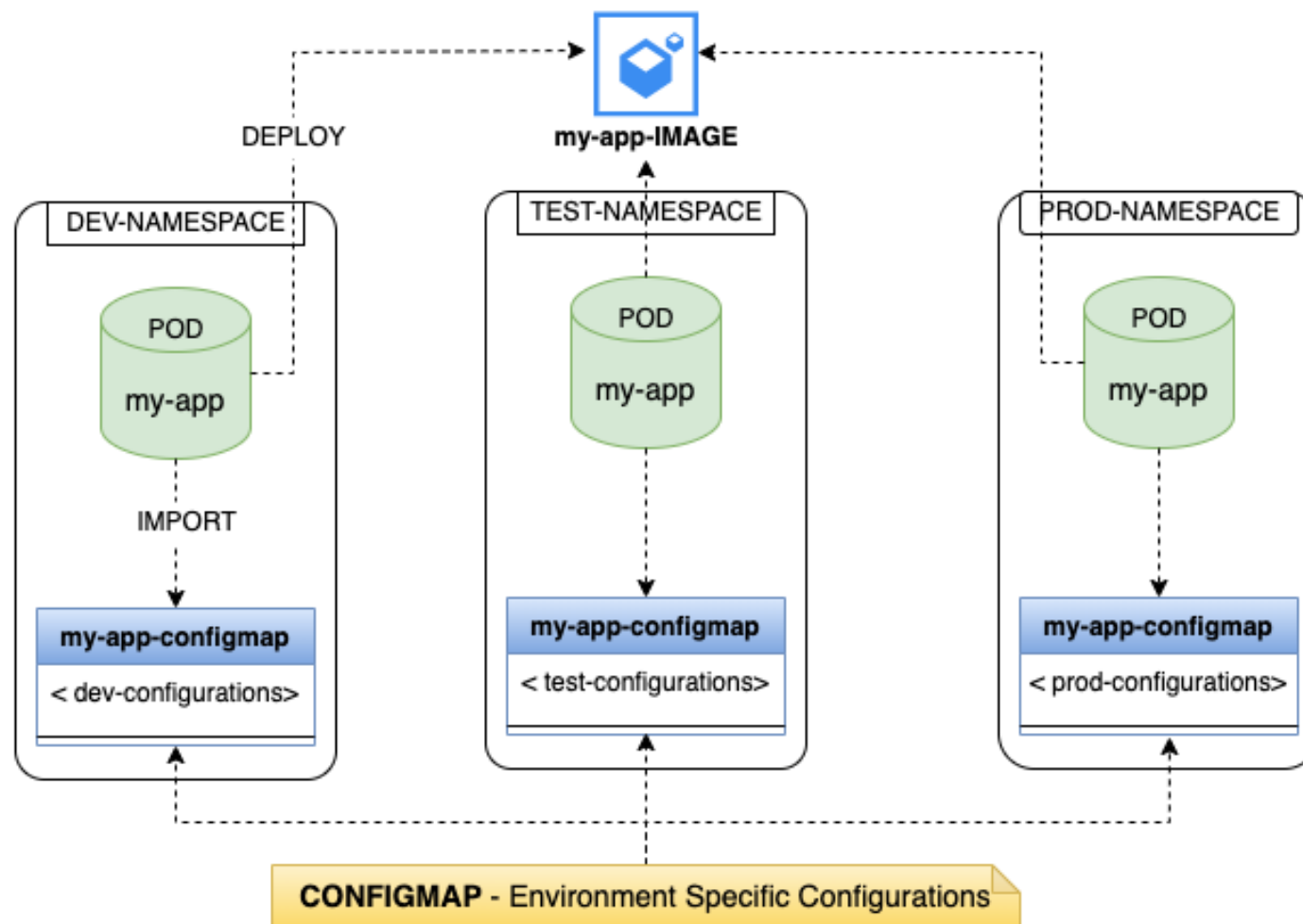


➤ Flannel (常用解决方案)



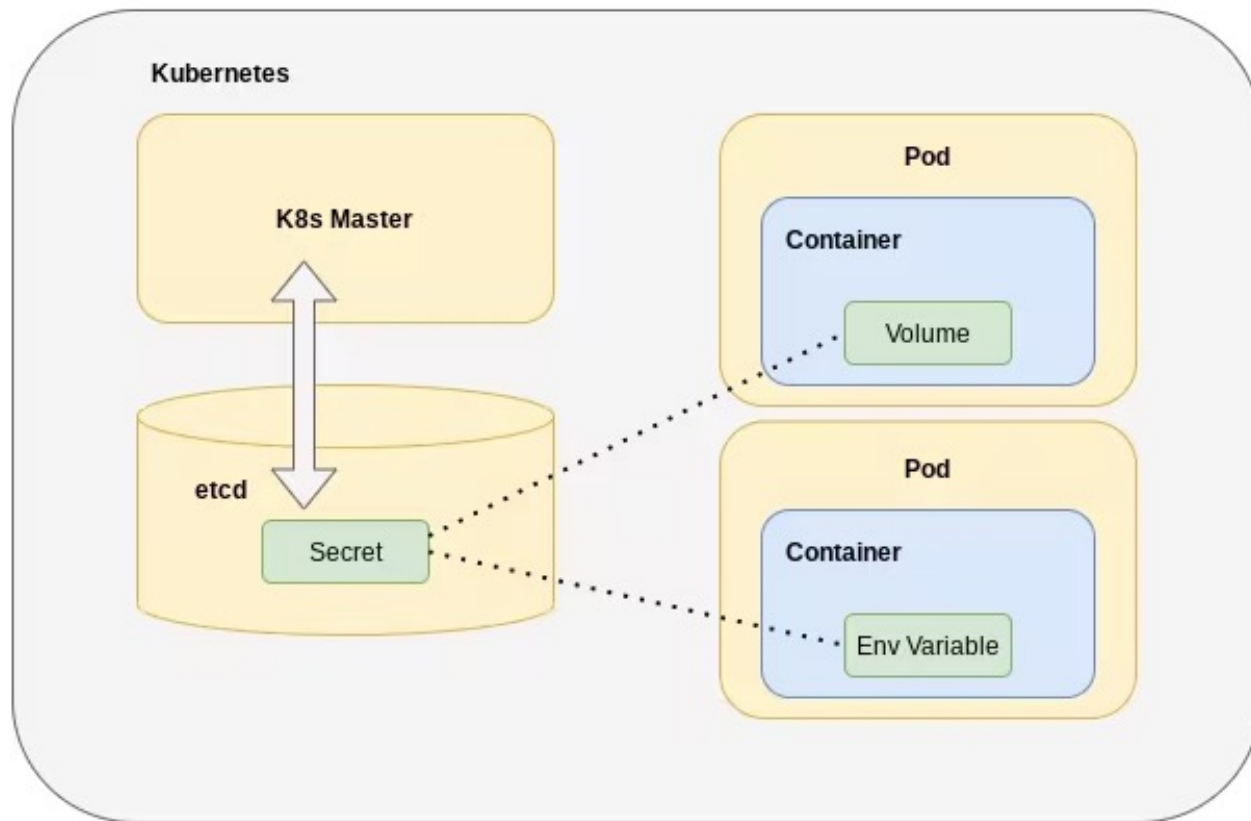
➤ ConfigMap

- ConfigMap API 提供了向容器中注入配置信息的机制，也可以用来保存整个配置文件，或者 JSON 二进制大文件。



➤ Secret

- Secret 解决了密码、token、密钥等敏感信息的配置问题，而不需要把这些敏感信息暴露到镜像或者 Pod Spec 中，Secret 可以以 Volume 或者环境变量的方式使用。



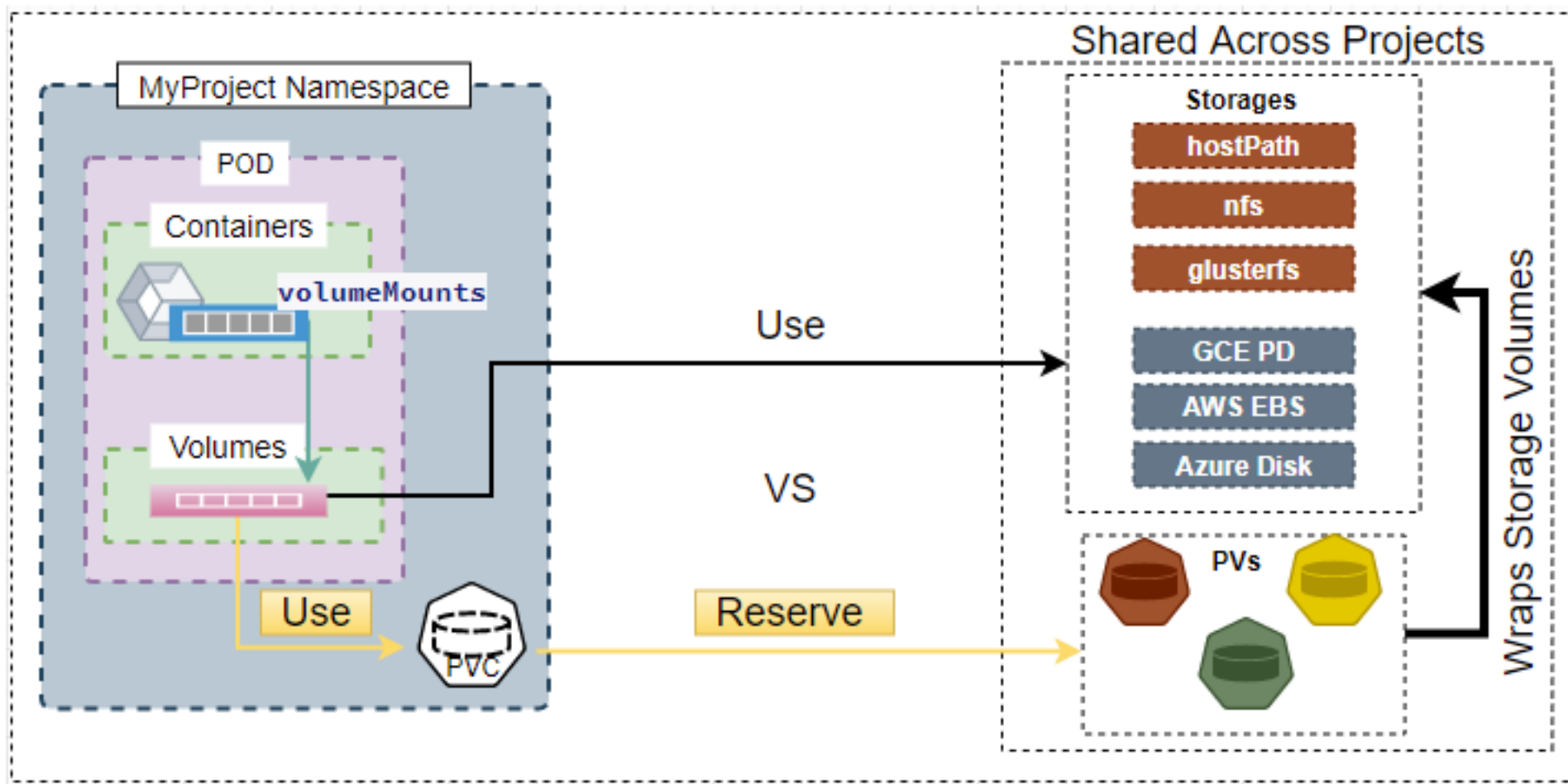
➤ Secret

- Opaque:
 - base64 编码格式的 Secret, 用来存储密码、密钥等; 但数据也可以通过 base64 – decode 解码得到原始数据, 所有加密性很弱。
- kubernetes.io/dockerconfigjson:
 - 用来存储私有 docker registry 的认证信息。
- kubernetes.io/service-account-token:
 - 用于被 serviceaccount 引用, serviceaccount 创建时 Kubernetes 会默认创建对应的 secret。Pod 如果使用了 serviceaccount, 对应的 secret 会自动挂载到 Pod 目录 /run/secrets/kubernetes.io/serviceaccount 中。

```
$ kubectl run nginx --image nginx
deployment "nginx" created
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-3137573019-md1u2             1/1     Running   0           13s
$ kubectl exec nginx-3137573019-md1u2 ls /run/secrets/kubernetes.io/serviceaccount
ca.crt
namespace
token
```

➤ Volume

- 容器卷的生命周期比Pod中所有容器都要长，当这个容器重启时数据得以保存。



➤ PV & PVC

- PersistentVolume (PV)

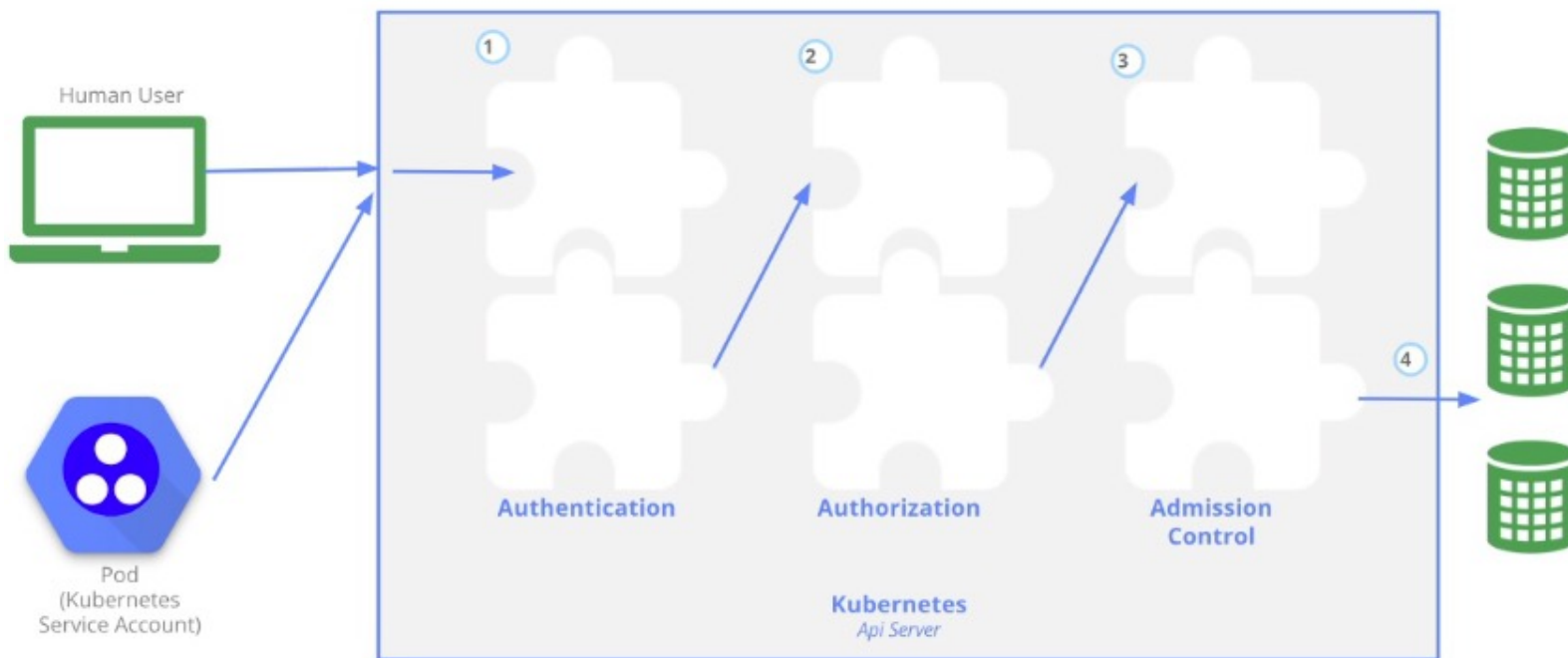
- 由管理员设置的存储，它是集群的一部分。就像节点是集群中的资源一样，PV 也是集群中的资源。PV 是 Volume 之类的卷插件，但具有独立于使用 PV 的 Pod 的生命周期。此 API 对象包含存储实现的细节，即 NFS、iSCSI 或特定于云供应商的存储系统。

- PersistentVolumeClaim (PVC)

- 用户存储的请求。它与 Pod 相似。Pod 消耗节点资源，PVC 消耗 PV 资源。Pod 可以请求特定级别的资源（CPU 和内存）。声明可以请求特定的大小和访问模式（例如，可以以读/写一次或只读多次模式挂载）

➤ 安全机制

- Kubernetes 是分布式集群的管理工具，保证集群的安全性是其重要的任务。API 服务器是中介通信组件内部的集群，也是外部控制入口。所以Kubernetes的安全机制基本围绕保护 API 服务器。
Kubernetes 使用认证（身份验证）、身份验证（授权）和准入控制（准入控制）来保证 API 服务器的安全。

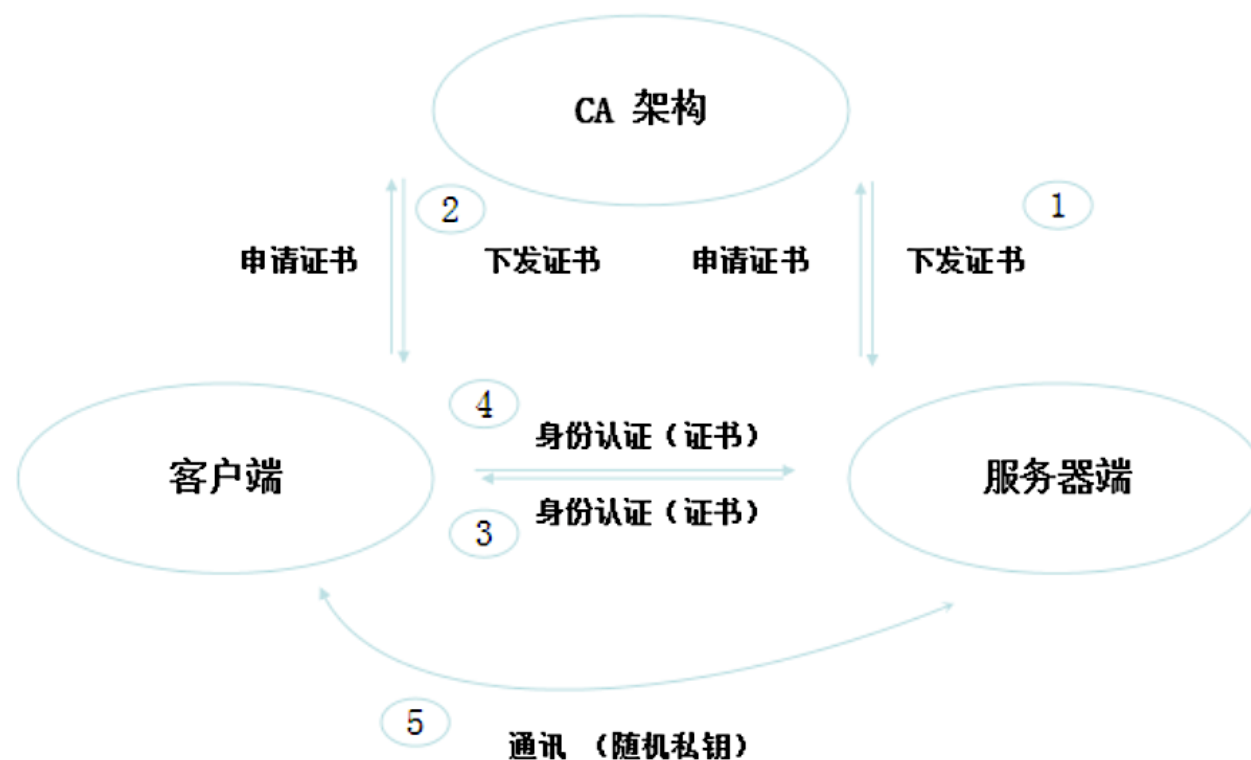


➤ Authentication

- HTTP Token 认证：通过一个 token 来识别合法用户。
 - HTTP Token 的认证是用一个很长的特殊编码方式的并且难以被模仿的字符串 – Token 来表达客户的一种方式。Token 是一个很长很复杂的字符串。每一个 Token 对应一个用户名存储在 API Server 能访问的文件中。当客户端发起 API 调用请求时，需要在 HTTP Header 里放入 Token。
- HTTP Base 认证：通过用户名 + 密码的方式认证。
 - 用户名 + 密码用 BASE64 算法进行编码的字符串放在 HTTP Request 中的 Heather Authorization 域里发送给服务端，服务端收到后进行编码，获取用户名及密码

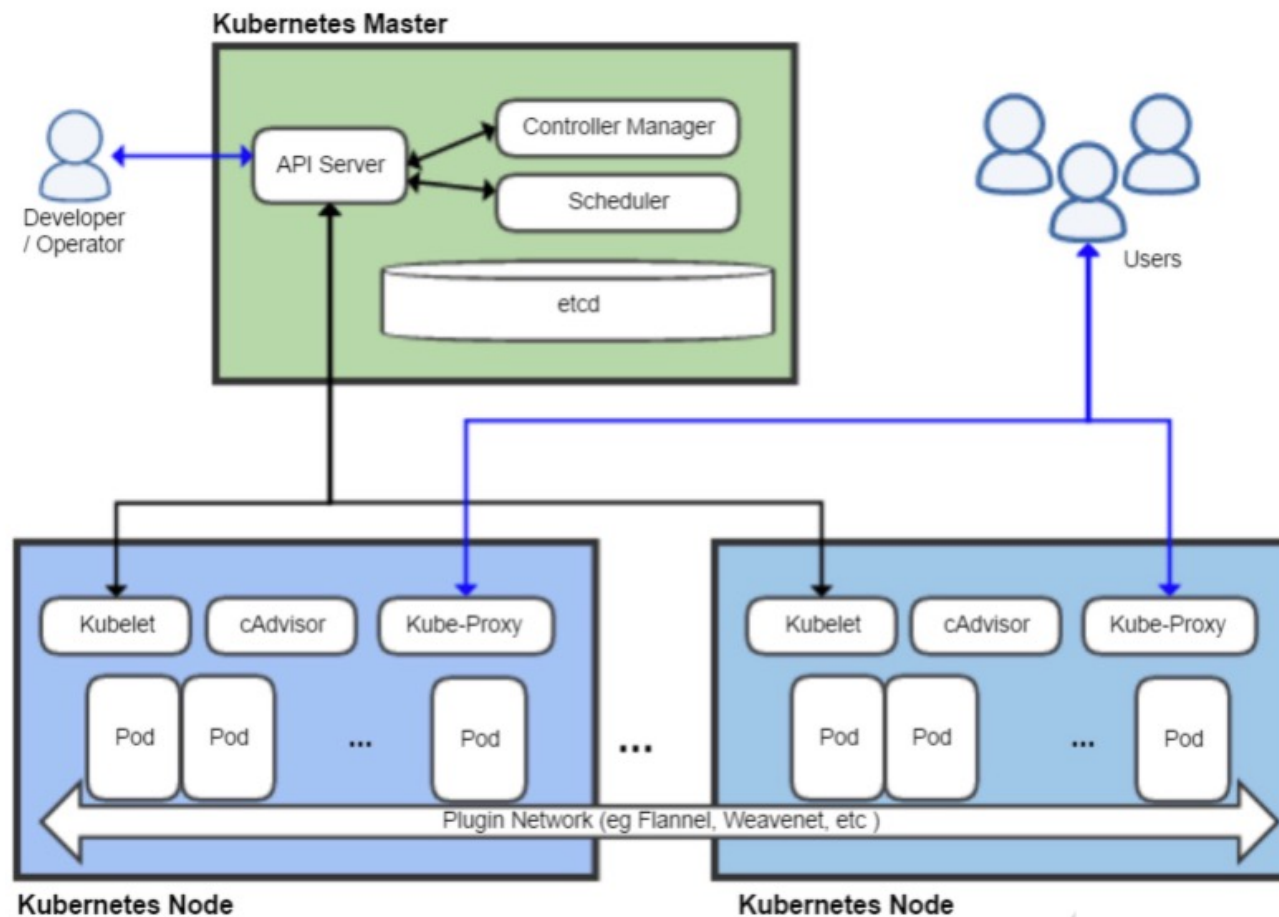
➤ Authentication

- 最严格的 HTTPS 证书认证：基于 CA 根证书签名的客户端身份认证反方式。



➤ Authentication

- 节点认证与用户访问



➤ Authentication

✓ 两种访问类型

- kubernetes组件对API Server的访问: kubectl、Controller Manager、Scheduler、kubelet、kube-proxy
- kubernetes管理的Pod对容器的访问: Pod(dashborad也是以Pod的形式运行)

✓ 安全性说明

- Controller Manager、Scheduler与API Server在同一台机器, 所以直接使用API Server的非安全端口访问, --insecure-bind-address=127.0.0.1
- kubectl、kubelet、kube-proxy访问API Server就都需要证书进行HTTPS双向认证

✓ 证书颁发

- 手动签发: 通过k8s集群跟ca进行签发HTTPS证书
- 自动签发: kubelet首次访问API Server时, 使用token做认证, 通过后, Controller Manager会为kubelet生成一个证书, 以后的访问都是用证书做认证

➤ Authentication

✓ kubeconfig

- Kubeconfig 文件包含集群参数(CA 证书、API Server 地址), 客户端参数(上面生成的证书和私钥), 集群 context 信息(集群名称, 用户名)。Kubernetes 组件通过启动时指定不同的 kubeconfig 文件可以切换到不同的集群。

✓ ServiceAccount

- Pod中的容器访问 API Server。因为 Pod 的创建、销毁是动态的, 所以要为它手动生成证书就不行了。Kubernetes 使用了 Service Account 解决 pod 访问 api server 的认证问题。

✓ Secret与SA的关系

- Kubernetes 设计了一种资源对象叫做 Secret 分为两类, 一种是用于 ServiceAccount 的 service-account-token, 另一种是用于保存用户自定义保密信息的 Opaque。ServiceAccount 中用到包含三个部分: Token、ca.crt、namespace。

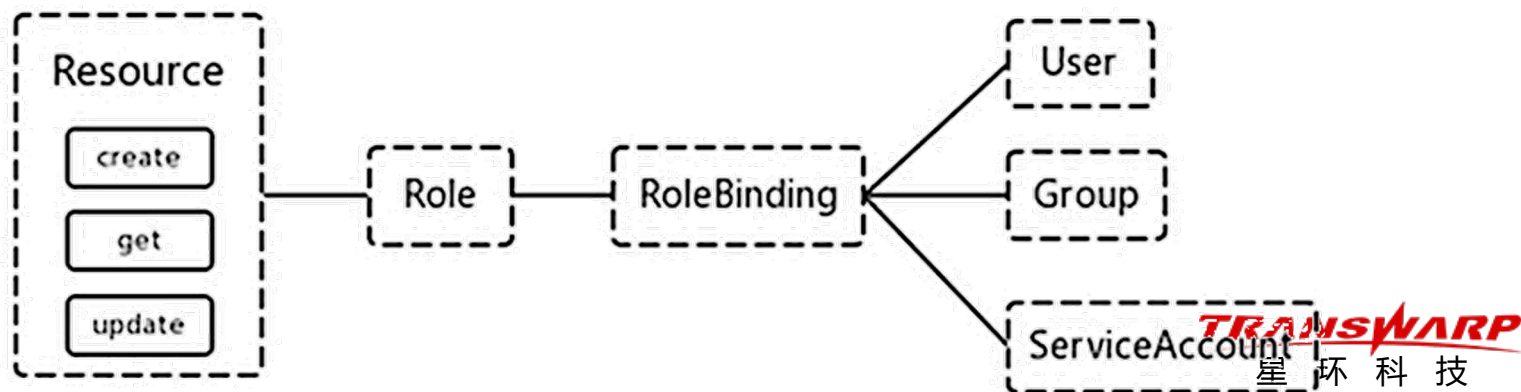
➤ Authorization

- 认证过程，只是确认通信的双方都确认了对方是可信的，可以相互通信。而鉴权是确定请求方有哪些资源的权限。API Server 目前支持以下几种授权策略（通过 API Server 的启动参数 “--authorization-mode” 设置）



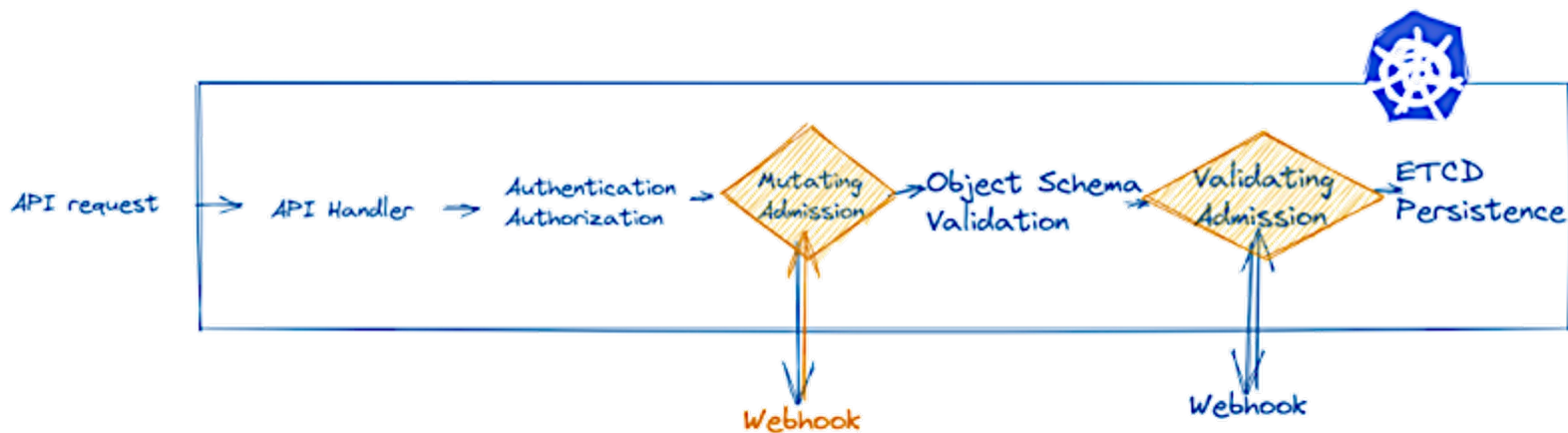
➤ Authorization

- ✓ AlwaysDeny: 表示拒绝所有的请求，一般用于测试
- ✓ AlwaysAllow: 允许接收所有请求，如果集群不需要授权流程，则可以采用该策略
- ✓ ABAC(Attribute-Based Access Control): 基于属性的访问控制，表示使用用户配置的授权规则对用户请求进行匹配和控制
- ✓ Webhook: 通过调用外部REST服务对用户进行授权
- ✓ RBAC (Role-Based Access Control) : 基于角色的访问控制，现行默认规则 (RBAC引入了4个新的顶级资源对象: Role、ClusterRole、RoleBinding、ClusterRoleBinding, 四种对象类型均可他通过 kubectl 与 API 操作)

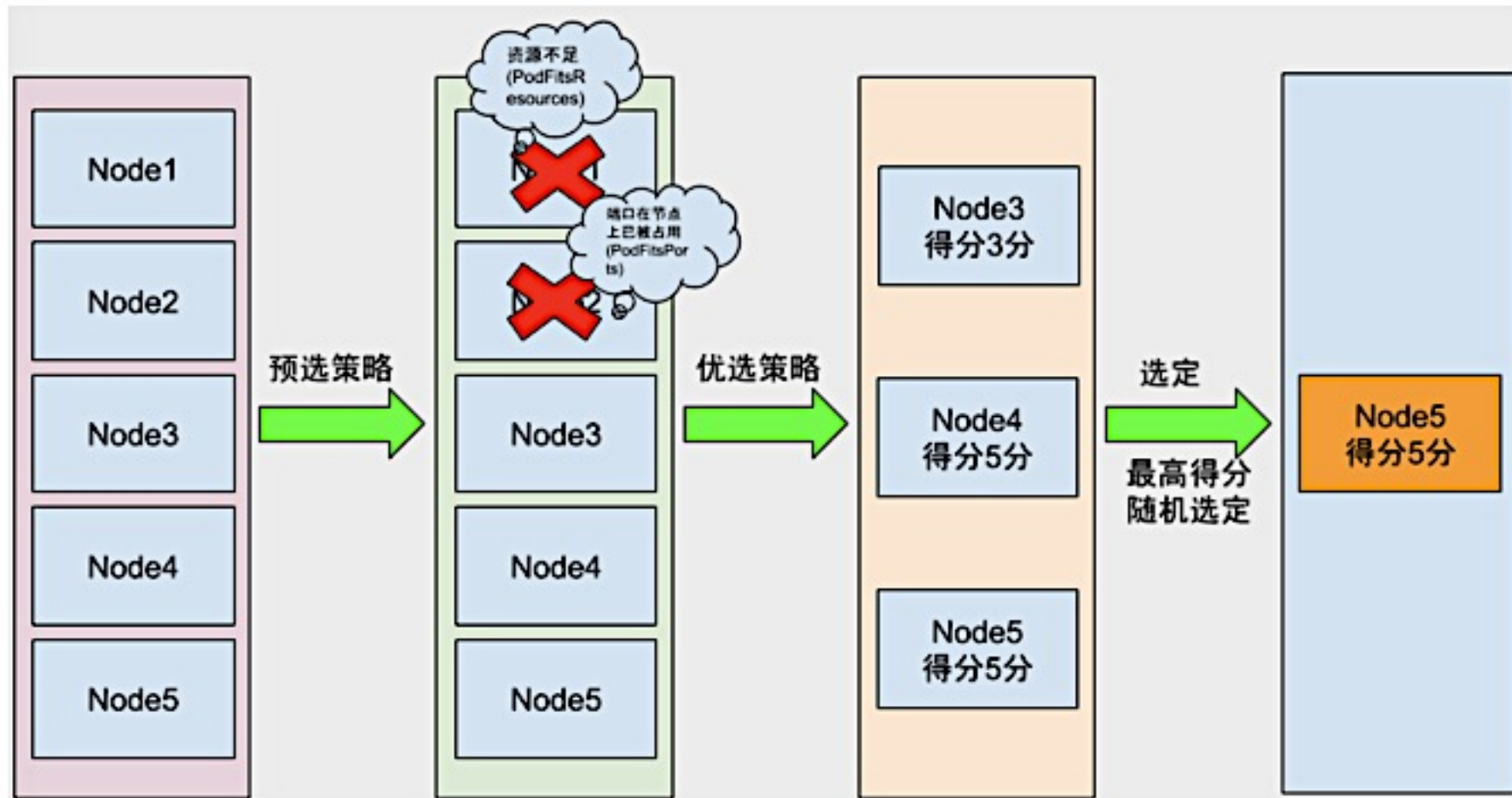


➤ Mutating Admission

- ✓ 准入控制是 API Server 的插件集合，通过添加不同的插件，实现额外的准入控制规则。甚至于 API Server 的一些主要的功能都需要通过 Admission Controllers 实现，比如 ServiceAccount
 - NamespaceLifecycle：防止在不存在的 namespace 上创建对象，防止删除系统预置 namespace，删除 namespace 时，连带删除它的所有资源对象。
 - LimitRanger：确保请求的资源不会超过资源所在 Namespace 的 LimitRange 的限制。
 - ServiceAccount：实现了自动化添加 ServiceAccount。
 - ResourceQuota：确保请求的资源不会超过资源的 ResourceQuota 限制。



➤ 调度策略



➤ 调度策略

- Scheduler 是 Kubernetes 的调度器，主要的任务是把定义的 pod 分配到集群的节点上。听起来非常简单，但有很多要考虑的问题：
 - 公平：如何保证每个节点都能被分配资源
 - 资源高效利用：集群所有资源最大化被使用
 - 效率：调度的性能要好，能够尽快地对大批量的 pod 完成调度工作
 - 灵活：允许用户根据自己的需求控制调度的逻辑
- Scheduler 是作为单独的程序运行的，启动之后会一直监听API Server，获取 PodSpec.NodeName为空的 pod，对每个 pod 都会创建一个 binding（必须遵守的），表明该 pod 应该放到哪个节点上。

➤ 常见的预选策略

- CheckNodeConditionPred: 检查节点是否正常
- GeneralPred: 通用预选策略, 包含多种策略:
 - HostName: (如果pod定义hostname属性, 会检查节点是否匹配。pod.spec.hostname)、
 - PodFitsHostPorts: (检查pod要暴露的hostports是否被占用。pod.spec.containers.ports.hostPort)
 - MatchNodeSelector: pod.spec.nodeSelector 看节点标签能否适配pod定义的nodeSelector
 - PodFitsResources: 判断节点的资源能够满足Pod的定义 (如果一个pod定义最少需要2C4G node上的低于此资源的将不被调度。用kubectl describe node NODE名称 可以查看资源使用情况)
- NoDiskConflict: 判断pod定义的存储是否在node节点上使用。(默认没有启用)
- PodToleratesNodeTaints: 检查pod上Tolerates的能否容忍污点 (pod.spec.tolerations)
- PodToleratesNodeNoExecuteTaints: Pod允许节点不执行污染; 默认不启用。
- CheckNodeLabelPresence: 检查节点上的标志是否存在 (默认没有启动)
- CheckServiceAffinity: 根据pod所属的service。将相同service上的pod尽量放到同一个节点 (默认没有启动)

➤ 常见的预选策略

- CheckVolumeBinding: 检查是否可以绑定 (默认没有启动)
- NoVolumeZoneConflict: 判断节点上部署Pod是否存在卷冲突。 (默认没有启动)
- CheckNodeMemoryPressure: 检查内存是否存在压力
- CheckNodeDiskPressure: 检查磁盘IO压力是否过大
- CheckNodePIDPressure: 检查pid资源是否过大
- MatchInterPodAffinity: 检查给定节点是否能够满足Pod对象的亲和性和反亲和性条件
- MaxEBSVolumeCount: 亚马逊弹性存储卷最大数量, 默认39
- MaxGCEPDVolumeCount: 谷歌容器引擎最大存储卷数量, 默认16
- MaxAzureDiskVolumeCount: Azure最大磁盘数量, 默认16

如果在 predicate 过程中没有合适的节点, pod 会一直在pending状态 (pending: 等待), 不断重试调度, 直到有节点满足条件。经过这个步骤, 如果有多个节点满足条件, 就继续 priorities 过程: 按照优先级大小对节点排序。

➤ 优选策略

- LeastRequested: 选择消耗最小的节点 (根据空闲比率评估 $\text{cpu}(\text{总容量} - \text{sum}(\text{已使用}) * 10 / \text{总容量})$)
BalancedResourceAllocation: 从节点列表中选出各项资源使用率最均衡的节点 (CPU和内存)
NodePreferAvoidPods: 节点倾向
- TaintToleration: 将pod对象的spec.toleration与节点的taints列表项进行匹配度检查, 匹配的条目越多, 得分越低。
- SelectorSpreading: 与services上其他pod尽量不在同一个节点, 节点上同一个service的pod越少得分越高。
- InterpodAffinity: 遍历node上的亲和性条目, 匹配项越多的得分越高
- NodeAffinity: 根据节点亲和性。
- MostRequested: 选择消耗最大的节点上 (尽量将一个节点上的资源用完)
- NodeLabel: 根据节点标签得分, 存在标签既得分, 没有标签没得分。标签越多 得分越高。
- ImageLocality: 节点上有所需要的镜像既得分, 所需镜像越多得分越高。 (根据已有镜像体积大小之和)

➤ 高级调度方式

- 当我们想把调度到预期的节点，我们可以使用高级调度分为：
 - 节点选择器: nodeSelector、nodeName
 - 节点亲和性调度: nodeAffinity
 - Pod亲和性调度: PodAffinity
 - Pod反亲和性调度: podAntiAffinity

➤ 自定义调度器

- 除了 Kubernetes 自带的调度器，你也可以编写自己的调度器。通过 spec:schedulename 参数指定调度器的名字，可以为 pod 选择某个调度器进行调度。比如下面的 pod 选择 my-scheduler 进行调度，而不是默认的 default-scheduler:

```
1  apiVersion: v1
2  kind: Pod
3  metadata:
4    name: annotation-second-scheduler
5    labels:
6      name: multischeduler-example
7  spec:
8    schedulername: my-scheduler
9    containers:
10     - name: pod-with-second-annotation-container
11       image: gcr.io/google_containers/pause:2.0
```



Q&A

TRANSWARP
星环科技